



FPGA-Accelerated Fractal Visualisation System

The Dream Team Report

June 2026

Anthony Bartlett CID: 02556059	Denzil Erza-Essien CID: 02593040	Lukas Mykhnenko CID: 02579708
Aadi Sharma CID: 02480177	Junjiang Wu CID: 02572610	Sam Snelson-Wash CID: 02599226

[GitHub Repository](#)

“Clouds are not spheres, mountains are not cones, coastlines are not circles, and bark is not smooth, nor does lightning travel in a straight line.”

— Benoit Mandelbrot

Department of Electrical and Electronic Engineering
Imperial College London

Contents

1	Introduction	3
2	Requirements and Specifications	4
2.1	Top-Level System Requirements	4
2.2	FPGA Accelerator Specifications	4
2.3	Custom Controller Specifications	5
2.4	Processing System Specifications	5
2.5	CPU Baseline Specifications	5
3	Project Management	6
3.1	Roles	6
3.2	Timeline	6
4	Overall System Design	7
4.1	User Interface	7
4.2	Data Flow and Component Interaction	7
5	Hardware Acceleration	8
5.1	Number Format Decision	8
5.2	Top-Level Design	8
5.2.1	Tile Scheduler	8
5.2.2	Iteration Core	9
5.2.3	Colour Palette	10
5.2.4	Pixel Write Engine	10
5.2.5	HDMI & PS Interaction	11
5.3	Resource Allocation	12
5.4	Timing Considerations	12
5.4.1	Reorder Buffer Timing Violations	12
5.4.2	Iteration Core Timing Violations	13
5.5	Version Releases	13
5.5.1	Version 1	13
5.5.2	Version 1.1	13
5.5.3	Version 2	14
5.5.4	Version 2.1	14
5.5.5	Version 3	14
5.6	Hardware Extensions	14
5.6.1	Dirty Rectangles	14
5.6.2	Progressive Rendering	14
5.6.3	Periodicity Checking	14
5.6.4	Mariani-Silver Algorithm	14
5.6.5	Scaled Perturbation	15
5.7	Testing	15
5.7.1	RTL Simulation & Verification	15
5.7.2	End-to-end System Testing	16
5.7.3	Processing-System Testing	16
6	Custom Controller	17
6.1	System Requirements	17
6.2	Breadboard Prototyping	17
6.2.1	Design Process	17
6.3	Hardware Design	17
6.3.1	PCB Design	17
6.4	Firmware Design	18
6.4.1	Buttons	18
6.4.2	Encoders	18
6.4.3	Joystick	19
6.4.4	Data Integrity	19
6.5	Testing & Verification Results	19
6.6	Case Design	20

7	Processing System	23
7.1	Role of the PS	23
7.2	Software Architecture	23
7.3	Input Abstraction and Controller Handling	23
7.4	Scene Management and Application State	24
7.5	FPGA Driver and Register Interface	24
7.5.1	View Parameter Conversion	24
7.5.2	Pixel Write Engine and Palette Control	25
7.6	Framebuffer Management and HDMI Output	25
7.7	HUD and Overlay Composition	26
7.8	Walkthrough and Free-Roam Modes	26
7.9	CPU Baseline Integration	26
8	CPU Baseline	28
8.1	Initial Design	28
8.2	Threaded Optimization	28
8.3	Timing & Analysis	28
9	Final Evaluation	29
10	Appendix	31
10.1	A: Set Calculations	31
10.1.1	Mandelbrot Set	31
10.1.2	Julia Set	31
10.1.3	Burning Ship Set	31
10.1.4	Tricorn Set	31
10.2	B: TMDS	32
10.3	C: Perturbation Theory	32
10.4	D: Redundant Blocks	32
10.5	E: Processing System	33
10.5.1	Full-PS Scenes	33
10.5.2	PL-Backed Scenes	34

1 Introduction

The main aim of this project was to design and build an educational visualisation tool capable of rendering computationally intensive mathematical functions in real time, and to showcase the advantages of FPGA hardware in achieving the level of responsiveness required for interactive use. Our chosen visualisation sets are Mandelbrot, Julia, Tricorn, and Burning Ship, all of which are well suited to hardware acceleration due to their "embarrassingly parallel" nature, each pixel's iteration count can be computed independently (see Appendix A).

Our system, which we call FractalScope, is composed of three closely integrated layers: a custom controller built around a Raspberry Pi Pico, a Python-based processing system running on the PYNQ-Z1's cores, and a SystemVerilog rendering accelerator implemented in the FPGA's programmable logic. Together, these layers form a complete pipeline from physical user input to real-time HDMI output, with each layer having a clearly defined responsibility and a well-specified interface to its neighbours.

This report covers the requirements and specifications we derived from the project brief, our approach to project management and team organisation, the overall system architecture and the reasoning behind its key design decisions, and detailed accounts of the hardware accelerator, custom controller, processing system, and CPU baseline implementations. It concludes with an evaluation of the complete integrated system against our stated requirements and how we feel our design measured up to our original goals as well as its overall effectiveness as an educational and interactive tool.

2 Requirements and Specifications

We derived the following requirements and specifications by expanding the project brief’s high-level objectives into system-specific and subsystem-specific requirements. Each requirement has also been given criteria to show how we can achieve each of these goals.

2.1 Top-Level System Requirements

	Requirement	Criteria
1	The system shall display a real-time interactive visualisation of a computationally intensive mathematical function.	Visualisation updates in response to user input with minimal perceptible lag at the display output.
2	The FPGA accelerator throughput shall exceed that of an equivalent CPU-only implementation.	Measured speedup is at least 10x compared to a CPU baseline
3	The system shall provide a physical user interface for intuitive adjustment of visualisation parameters.	A user unfamiliar with the system can navigate the visualisation without written instructions.
4	The system shall display real-time visualisation state as a HUD overlay.	Zoom level, iteration depth, frame rate, and current coordinates are visible on screen during operation.

Table 1: Top-level system requirements

2.2 FPGA Accelerator Specifications

	Requirement	Criteria
1	The accelerator shall support multiple fractal sets as selectable runtime modes.	All sets render correctly, with the Julia set using a register-supplied complex constant.
2	The accelerator shall support a configurable maximum iteration depth adjustable via register interface without recompilation.	Iteration count is adjustable across a range defined by the PS at runtime
3	The accelerator shall use Q4.22 fixed-point arithmetic.	No visible precision artefacts are present within the operational zoom range.
4	The accelerator shall exploit parallelism through multiple simultaneous iteration cores.	A minimum of 16 cores are active during rendering.
5	The accelerator shall output a continuous HDMI video stream.	No frame tearing is observable during parameter updates.
6	The accelerator shall expose all view parameters via an AXI-Lite register interface.	Centre, zoom, and iteration depth are writeable from the processing system.

Table 2: FPGA accelerator requirements

2.3 Custom Controller Specifications

	Requirement	Criteria
1	The controller shall transmit structured input packets to the processing system over USB at 100 Hz.	Packets are received and parsed at 100 Hz with zero CRC failures under normal operating conditions.
2	The controller shall include two rotary encoders for zoom and iteration depth adjustment.	Each encoder registers the correct signed delta.
3	The controller shall include a joystick for panning, with auto-calibration on power-on.	The joystick produces a zero-centred output after calibration and full deflection reaches ± 1968 ADC counts.
4	All push buttons shall be debounced in firmware with a 5 ms threshold.	Each button event triggers exactly once per physical click with no chatter observed on press or release.
5	Every transmitted packet shall carry a CRC8-CCITT checksum.	The processing system detects and silently drops corrupted packets without causing erratic navigation.
6	The controller shall be housed in a custom 3D-printed casing.	All inputs are accessible through the enclosure and the PCB sits on standoffs without requiring direct fastening to the casing.

Table 3: Custom controller requirements

2.4 Processing System Specifications

	Requirement	Criteria
1	The processing system shall parse controller packets and translate them into FPGA register writes.	View parameters update on the FPGA in response to each controller event.
2	The processing system shall composite a HUD overlay onto each rendered frame before HDMI output.	The overlay is legible at 720p without visibly degrading frame rate.
3	The system shall fall back to keyboard input if the controller is disconnected.	Disconnecting the controller mid-session does not crash the application and keyboard navigation remains functional.
4	The processing system shall support selectable fractal sets at runtime.	The user can switch between modes via controller input without restarting the application.

Table 4: Processing system requirements

2.5 CPU Baseline Specifications

	Requirement	Criteria
1	A CPU-only implementation shall be produced for benchmarking.	The CPU baseline renders a complete frame at 720p and produces a correct output image.
2	The CPU baseline shall use multi-threading across both PYNQ CPU cores.	Measured throughput exceeds that of the single-threaded implementation.
3	End-to-end frame latency shall be measured for both the CPU baseline and FPGA accelerator under identical conditions.	Latency figures are recorded and presented comparatively in the evaluation.

Table 5: CPU baseline requirements

3 Project Management

The main focus early in the project window was to create clear outlines of each member’s roles, main targets and milestones, as well as time frames in which we felt these milestones could comfortably be met while still producing a high quality and polished product.

3.1 Roles

Initially, we defined our roles which were decided by two main factors:

- Determining roles by our strongest areas.
- Allowing cooperation and overlap between members

We eventually concluded with the following roles which we felt allowed each member to work on areas they enjoyed and were good at while also allowing some flexibility in order to encourage cooperation and working as a team.

Team Member	Role
Sam	Project Manager / Controller Design
Anthony	Processing System / Hardware
Denzil	CPU Baseline / Hardware
Junjiang	User Interface / CPU Baseline
Lukas	Analogue Controller Design
Aadi	Digital Controller Design

Table 6: Team Member Roles

We elected to have regular whole group meetings weekly to ensure everyone always knew what each other was working on and so no singular member felt that they couldn’t ask for advice or assistance when needed along with regular communication using other platforms.

3.2 Timeline

Next, we had to choose a suitable timeline in which we wanted to adhere to. Doing this would give every member clear main goals which could be further broken down personally in order to make sure each stage was completed by it’s respective date.

The main project timeline is as follows:

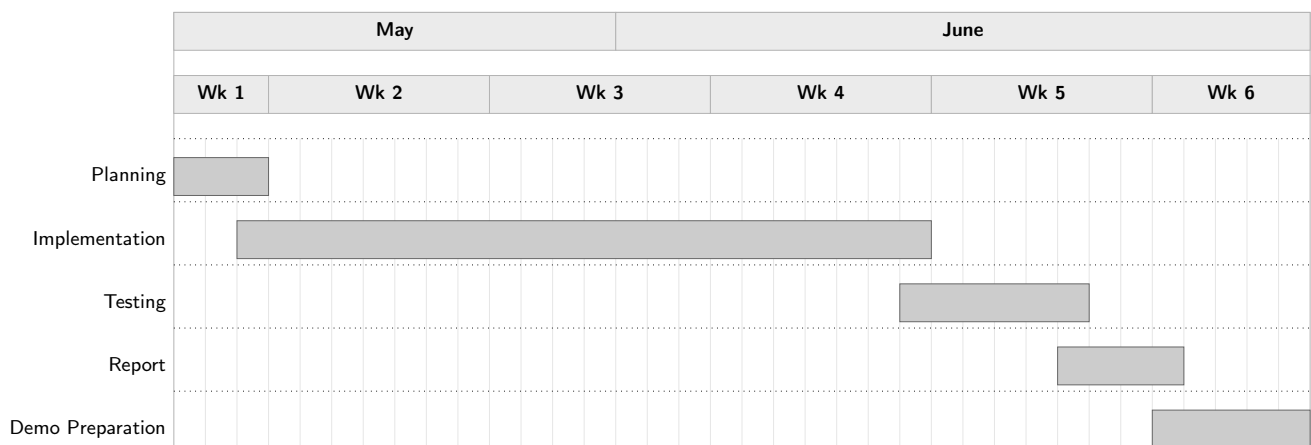


Figure I: Gantt Chart representation of the project timeline phases.

As previously mentioned, these time frames were declared as to allow each member to produce quality work whilst not struggling with harsh deadlines, ultimately in order to produce a high quality final design.

4 Overall System Design

For our overall architecture, one key decision was the separation of input, application logic, and rendering into distinct layers.

The custom controller is responsible only for reading physical inputs and transmitting raw event packets to the processing system.

The processing system receives these events and interprets them according to the current scene, translating them into parameter changes which are then written to the FPGA via AXI-Lite registers.

The PL computes fractal pixel colours and writes them to DDR

4.1 User Interface

When it came to our UI, we decided to implement a custom physical controller and a HUD-style overlay.

A physical controller would allow the user to intuitively adjust the parameters of our visualisation. We made this decision as exploring the fractal would mean continuously varying parameters: zoom level, iteration, and the complex plane coordinates of the view centre. These map naturally onto analogue inputs such as rotary encoders, with the joystick providing intuitive control over panning. This allows the user to navigate the fractal smoothly, and provides an interesting way to interact with the visualisation.

This means the display itself only requires a lightweight HUD overlay, leaving the full framebuffer available for the FPGA-rendered fractal image. The overlay presents real-time system information including zoom level, iteration depth, and frames per second, directly serving the educational goal of the project by allowing the user to observe in real time how the hardware parameters affect the visualisation.

4.2 Data Flow and Component Interaction

The data flow through the FractalScope follows a strict pipeline, with each component having a clearly defined role and interface. Figure II shows the overall flow from user input to display output.

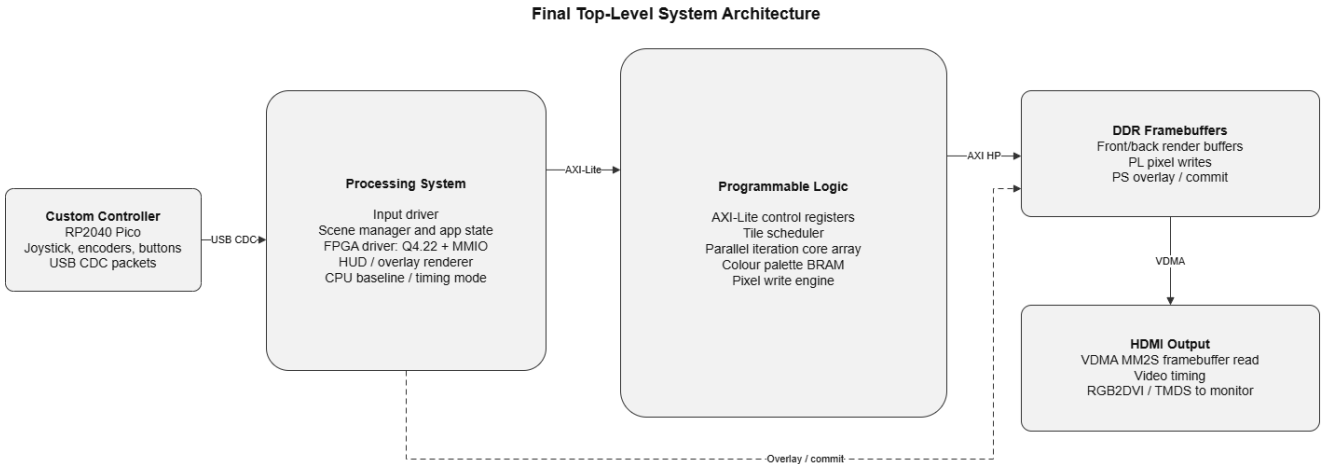


Figure II: Top-Level System Architecture

At the start of each frame, the controller driver reads the latest packet from the RP2040 over USB and updates the application state. The application state holds all the parameters needed to describe the current view. The scene manager then interprets the input in the context of the current mode and applies the appropriate changes to the application state.

The FPGA driver takes the updated application state and converts the floating-point view parameters into Q4.22 fixed-point register values, computing the top-left coordinate and per-pixel step size from the centre and zoom. These values are written to the FPGA via AXI-Lite registers, and the render pipeline is started.

While the FPGA renders into a back framebuffer, the processing system waits for the Pixel Write Engine to report completion. The PS then invalidates the PL-written buffer, expands any sparse progressive pass, composites the HUD, flushes the completed image, and commits it to the fixed framebuffer read by the VDMA. In the final demo this fixed-display commit-copy path was preferred over direct VDMA frame-store swapping because it avoided the left-edge wrap-around and tearing observed during earlier integration tests.

5 Hardware Acceleration

5.1 Number Format Decision

The fractal datapath uses a signed fixed-point number format rather than floating point. This was chosen because the main iteration calculation is repeated many times across many parallel cores, so the arithmetic must be efficient in FPGA resources. Floating-point arithmetic would provide a wider dynamic range, but it would require significantly more logic and DSP resources, reducing the number of iteration cores that could fit on the device. The final design uses a signed Q4.22 format for the complex coordinates and intermediate iteration values. This gives a total width of 26 bits, with 22 fractional bits. The representable range is approximately:

$$[-8 \leq x < 8]$$

and the resolution is:

$$[2^{-22} \approx 2.38 \times 10^{-7}]$$

This range is sufficient for the fractal views used in the project. The Mandelbrot set is normally viewed within a region roughly spanning (-2.5) to (1.5) on the real axis and (-1.5) to (1.5) on the imaginary axis, so Q4.22 provides enough integer range while preserving fine fractional precision for zooming.

At 1280 pixels wide, this means that once the view approaches approximately $1280 \times 2^{-22} \approx 3.05 \times 10^{-4}$, neighbouring pixels begin to approach the fixed-point resolution limit. The chosen format is therefore suitable for interactive exploration, but not for deep zooming.

The iteration core performs complex squaring, so the most important operation is fixed-point multiplication. When two Q4.22 values are multiplied, the raw product has twice the bit width and twice the number of fractional bits. The result is then shifted back by 22 bits to return to Q4.22 format. This ensures all stages of the iteration pipeline use the same coordinate format, simplifying the design and avoiding repeated conversions between formats.

The number format was therefore a compromise between precision, zoom depth, timing, and resource usage. Increasing the number of fractional bits would allow deeper zooms before precision artefacts appear, but it would also increase multiplier width and place more pressure on DSP usage and timing. Reducing the width would save resources, but would make the fractal visibly inaccurate at moderate zoom levels. Q4.22 was chosen because it gave enough precision for interactive exploration while still allowing the design to keep multiple pipelined iteration cores operating in parallel.

Overall, fixed-point Q4.22 gave the best balance for the current hardware accelerator. It keeps the arithmetic deterministic, resource-efficient, and fast enough for the parallel PL pipeline, while still giving the PS a simple way to convert user-facing floating-point view parameters into hardware register values.

5.2 Top-Level Design

The system follows the diagram shown in Figure III.

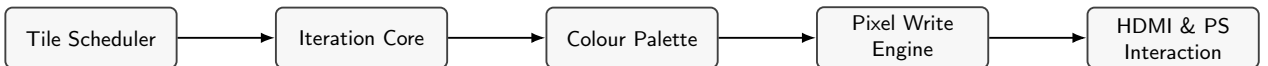


Figure III: System Architecture for Hardware Acceleration

We split the design into these specific modules as they all have separate and significant functions. By splitting them up in this way, we were also able to effectively debug through verification.

5.2.1 Tile Scheduler

The tile scheduler block is in control of taking coordinate data from the PS and forming rectangle blocks (tiles) of the entire 1280x720 screen, sending pixels of these tiles to a specific core allowing for the iterative calculation of the coordinate represented by that pixel. This block also includes a tile evaluator which allows us to scale these tiles given the stage of progressive rendering we are at, and confirm the tile fill of a region which is positively evaluated by the Mariani-Silver (M-S) algorithm.

The tile scheduler architecture follows that of the diagram in Figure IV.

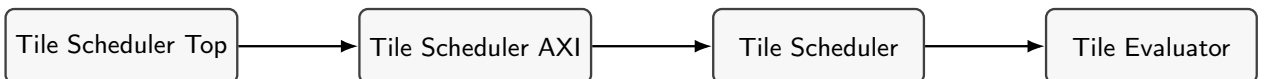


Figure IV: Tile Scheduler System Architecture

In the AXI block, we use the Vivado generated AXI4 Peripheral, allowing us to collect data controlled in the PS such as the max iterations, the centre values, and the Julia set specific values. This is all packed as an AXI-Lite Slave port which can easily be controlled in the PS given the block appears as memory-mapped input/output (MMIO).

The tile scheduler takes in the parameters for pixel coordinate generation, Julia set calculation, Iteration Core logic, and Colour Palette signals whilst outputting a continuous stream of data for all the cores within the Iteration Core block to do specific set calculation or fill a rectangle through the M-S algorithm.

The tile scheduler passes the inputs from the Iteration Core block, along with some internal evaluation signals to the tile evaluator. It then skips the static UI overlay pixels, selects the mode of calculation, and chooses the core to write data into through a valid/ready handshake. Finally, the scheduler tracks its progress using a dedicated state machine, which continuously evaluates whether the current coordinate resides on the active tile's perimeter, within its interior, or if the system must advance to the next tile.

The tile evaluator is integral to the M-S algorithm, continuously determining the outputs of the Iteration Core block, calculating how many of the perimeter pixels are equal to each other. If this is the case, the block will raise a signal to indicate that this rectangle should be filled with the corresponding iteration value from Iteration Core, causing the scheduler to enter the FILL state. This allows us to bypass the pixel-by-pixel calculation that would be done otherwise, passing this signal straight to the colour palette in a single-cycle delay breaking combinational timing paths.

5.2.2 Iteration Core

The Iteration Core block is responsible for our cross-multiplication operations to calculate each iteration of the fractal sequences, returning the escaped iteration value. This block is heavily pipelined and optimised to ensure fair selection and efficient calculation of a heavily parallelised calculation design.

The Iteration Core architecture follows that of the diagram in Figure V

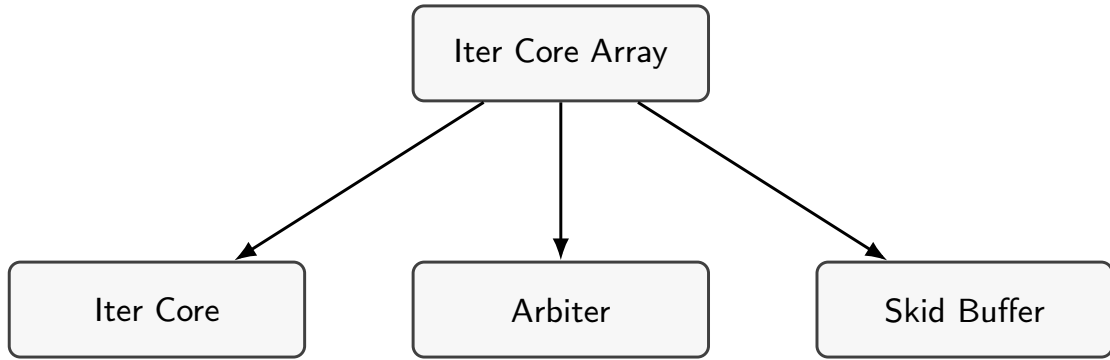


Figure V: Iteration Core System Architecture

The Iter Core Array block acts as a top module which primarily focuses on taking all the data from the tile scheduler for its 23 cores slicing relevant bits and feeding them into single iter core blocks. It packs the output data from each iter core block and packs it into an array of data for the skid buffer to temporarily hold. This data is then sent to the arbiter and back into another skid buffer to act as a pipeline stage. The outputs from this block are sent to the colour palette for colour mapping, as well as the tile evaluator for M-S implementations.

The Iter Core block is the key block in our complex fractal operations, acting as a 7-stage pipelined fixed-point calculator. It is given the mode that it is required to calculate and will then do this calculation through a number of pipelined stages. In this block, a mix of sequential and combinational logic is used intentionally to ensure that timing constrictions are met. For example, trivial calculations, such as mode selection, or bit slicing is purely combinational whereas logic requiring deeper complexity for the routing algorithms is done through sequential logic. Within this sequential logic, we have 3 cross-multiplications operations where the products of the complex squared terms are deduced. This requires 9 DSP slices for each iter core block. Before all pipeline transfers, the 7th stage values are passed back as outputs to iter core array.

The skid buffer block acts as a 2-cycle delay buffer for outputs in both the iter core and arbiter blocks, breaking apart the long combinational paths in the AXI-Stream valid and ready handshakes. It uses a FIFO policy to output data according to its relative time to enter the buffer.

The arbiter takes in data from multiple iteration cores and multiplexes the data paths outputting data from a core. This block acts as a round-robin scheduler which is useful as it maximizes throughput while also being fair preventing pipeline starvation of cores. This block makes two priority trees, selecting the highest priority core and outputting it through combinational logic, sending it to the final skid buffer to output relevant data to the colour palette.

5.2.3 Colour Palette

The Colour Palette block is responsible for converting the iteration results from the Iteration Core into visible RGB pixel values. The iteration core only tells us whether a point escaped, the iteration count at which this happened, and the corresponding pixel sequence number. This information is not directly suitable for HDMI output, so the colour palette maps the numerical iteration data into an 8-bit red, green, and blue value which can then be passed to the Pixel Write Engine.

The Colour Palette architecture follows that of the diagram in Figure VI

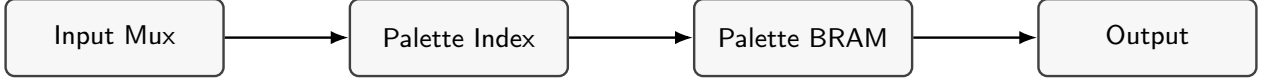


Figure VI: Colour Palette System Architecture

The input mux is required because the colour palette can receive two slightly different types of pixel result. Under normal rendering, the input comes from the Iteration Core Array after a pixel has been individually calculated. However, when the Mariani-Silver algorithm positively evaluates a tile region, the tile scheduler can send a tile-fill result directly into the colour palette. This allows a region to be filled using one representative iteration value instead of calculating and colouring every pixel independently.

The palette block preserves the pixel sequence number throughout the colour-mapping process. In the final framebuffer architecture, this sequence number is treated as the pixel index. This is important because the Pixel Write Engine writes pixels directly into DDR memory using their framebuffer position. Therefore, even if pixels complete out of order, each coloured result still carries the information needed to write it to the correct memory address.

The core of the module is a BRAM-backed lookup table containing eight palettes. Each palette contains 1024 RGB entries, giving a total of 8192 stored 24-bit colour values. The palette is selected at runtime using a 3-bit `palette_select` signal from the PS. This allows the user to cycle between different colour schemes without regenerating the FPGA bitstream.

Before the lookup occurs, the raw iteration count is scaled into a 10-bit palette index. This is needed because the maximum iteration count can be changed during runtime. If the iteration count were used directly, increasing the maximum iteration depth would compress most visible colours into only part of the palette range. Instead, the PS provides a `palette_scale` value which spreads the active iteration range across the full 1024-entry palette.

The colour palette uses registered pipeline stages to avoid a long combinational path between the Iteration Core and the Pixel Write Engine. The first stage calculates the scaled palette address, the second stage performs the BRAM lookup, and the final stage registers the RGB output. The valid/ready handshake is preserved across these stages, so back-pressure from the Pixel Write Engine can safely stall the palette without losing the corresponding pixel index or iteration metadata.

Since the palette index and scale are controlled from the PS, the final system can change both iteration depth and colour scheme interactively while keeping the high-throughput colour mapping inside the FPGA.

5.2.4 Pixel Write Engine

The Pixel Write Engine block is a custom Direct Memory Access (DMA) back-end for the fractal pipeline. It accepts the RGB colour data generated by the colour palette and directly writes calculated pixels into the PS DDR memory bypassing the CPU.

Given the use of the M-S algorithm, which fills tiles spatially in contrast to the raster scan used in previous implementations, we chose to implement pipelined outstanding AXI writes through a synchronized internal FIFO to buffer incoming pixels with their corresponding DDR memory address.

The Pixel Write Engine architecture is shown in Figure VII.

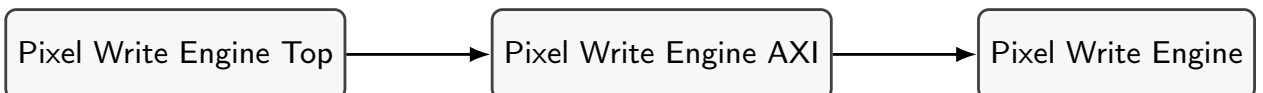


Figure VII: Pixel Write Engine System Architecture

The AXI block is used to output master signals to the DDR (through the HP port of the Zynq processing system). The master signals are calculated in the pixel write engine block and are passed to the slave port of the processing system in accordance with its valid/ready handshake.

The pixel write engine includes a FIFO buffer holding the colour of the pixels as well as their corresponding address value. It can then be split into two parallel channels which are mapped to AXI4 channels. Channel A is the AXI Issue engine (AW & W channels) where sequential logic controls the FIFO and AXI connection filling the FIFO

with relevant colours for the pixels and their addresses and the AXI will pull this data from the FIFO if there is room and the AXI isn't currently sending any data. This engine is capable of keeping up to 30 outstanding write requests simultaneously. Channel B is the AXI response engine, i.e. the connection between the AXI and DDR where written responses are asynchronously tacked. We send values such as the number of pixels written and the number of errors which occurred in the data transfer. This became integral for debugging when looking at our fractal images and testbenches. Finally, when all the data is sent, the done signal goes high indicating the end of the data transfer.

5.2.5 HDMI & PS Interaction

The HDMI pipeline takes the 32-bit RGB stream written into the PS DDR memory and delivers data to the FPGA pins used to produce an image on a screen through a HDMI port. The HDMI pipeline has the following architecture shown in Figure VIII:

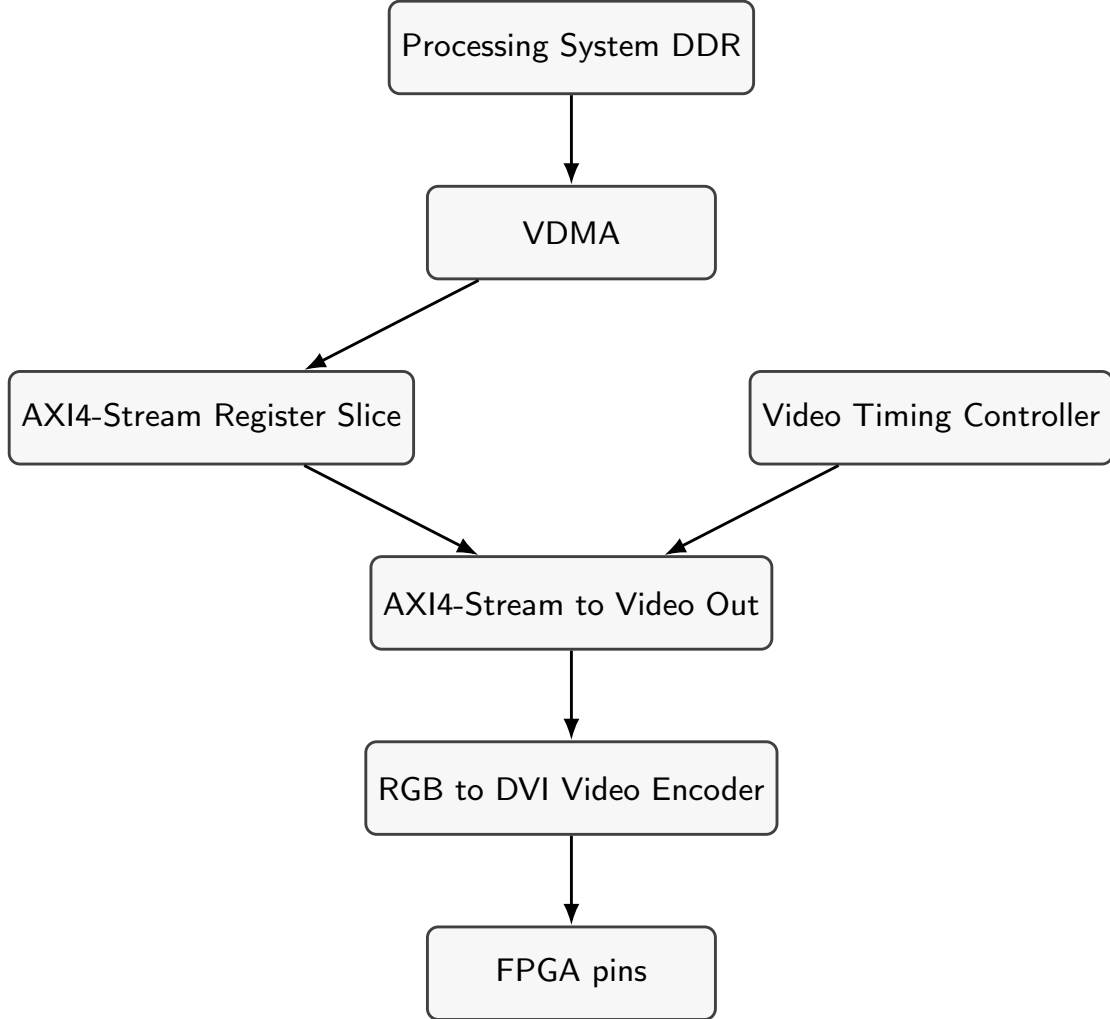


Figure VIII: HDMI Pipeline Architecture

The 32-bit RGB stream is first delayed by one cycle via the AXI4-Stream Register Slice which acts as a pipeline buffer mitigating timing issues breaking apart heavy combinational logic paths from the AXI-Stream between DDR and the Video Direct Memory Access (VDMA) module.

The VDMA reads the data from the DDR, holding it in an internal asynchronous FIFO and sends a continuous synchronous AXI4-Stream. We also convert our 32-bit RGB data into a 24-bit signal through the AXI Stream Subset converter as each RGB colour only requires 8-bits per colour.

The video timing controller is used to ensure our required resolution and frame rate can be achieved from our design. In order to obtain a frame rate of 60 fps we use the following equation:

$$f_{pixel} = TotalWidth \times TotalHeight \times fps$$

$$f_{pixel} = 1650 \times 750 \times 60 = 74.25MHz$$

This gave us a pixel clock frequency of 74.25 MHz which we use throughout this HDMI pipelined system.

We pass these signals to the RGB to DVI encoder, which has been taken from the Digilent Vivado Library, allowing for RGB data to be parallelised and encodes the 8-bit colour signals into 10-bit symbols through Transition Minimized Differential Sampling (TMDS).

As TMDS is a serial signalling scheme, it requires a serial clock to ensure that it can sample the signal effectively. Given that it outputs a singular bit per clock edge, (and HDMI takes data at both the rising and falling edges), the serial data needs to be taken at five times the rate of the pixel data, hence giving a serial clock of 371.25 MHz. The TMDS system sends dual-wire differential signals ensuring our final signal is immune to noise (see Appendix B), and sends these signals to the mapped pins on the FPGA.

5.3 Resource Allocation

In designing our fractal set, we understood that the FPGA would handle all of our Multiply-Accumulate (MAC), operations given that its DSPs can run a larger number of operations in a single clock cycle simultaneously without the complexity and overhead in preventing race conditions that software deals with. Therefore, one target we had set out was to try and maximise our DSP utilisation to ensure that these heavy operations were calculated as efficiently as possible.

In table 7, we have presented our resource utilisation summary for version 3 of our hardware design. In this table we can see that 212 out of 220 DSP slices were used. As mentioned in Iteration core, each core requires 9 DSP slices to operate, hence the idea of using 24 cores was proposed (where allocation would spill to the LUTs and FFs once DSP utilisation is fully exhausted). However we ultimately chose to go against this proposal given that our LUT utilisation would reach over 80% causing congestion on the board. Not only did this have negative effects on our timing (detailed in Timing Considerations), but this higher utilisation caused an increased power consumption through an increase in parasitic capacitance increasing dynamic power draw and heat. Given we wanted to ensure our system was both efficient and sustainable, we decided to keep our number of cores to 23.

Resource	Utilization	Available	Utilization (%)
LUT	34536	53200	64.92
LUTRAM	6342	17400	36.45
FF	35766	106400	33.61
BRAM	9	140	6.43
DSP	212	220	96.36
IO	8	125	6.40
BUFG	4	32	12.50
MMCM	1	4	25.00

Table 7: FPGA resource utilization summary for v3.

5.4 Timing Considerations

In our goal to produce fractal sets with the lowest latency, we often ran into issues with timing due to multiple timing violations. In our v1 versions, we often had issues with the reorder buffer block. Later versions would have timing violations occur in the iteration core block in which we used multiple approaches to resolve both these violations without heavily impacting our final design.

5.4.1 Reorder Buffer Timing Violations

Within our reorder buffer design, we planned to send an ordered burst of pixels with their respective exit iteration values, holding any pixels which had returned earlier than others from the iteration core block in a buffer. In version 1.1, we increased this buffer size to 8192. This would lead to timing violations as we attempted to pass multi-bit signals such as the iteration value, the sequence number of the pixel, and other signals to acknowledge the escaping of the coordinate represented by the pixel.

Given that we not only had to pass these variables, but also look through the dual-port buffer with reading and writing indexes checking for values and buffer space respectively, all in one cycle, it was obvious to us, and then later proved, that this would be a significant timing violation. In addition to this, as our design required chronological transmission of pixels, this buffer was necessary, yet its minimal size relative to the number of pixels we sent caused issues with our end-to-end latency and throughput.

In order to resolve this issue, we end up making the reorder buffer redundant (see Appendix D). We realised that this reorder section was extremely inefficient and was the critical path in our timing analysis. Instead of reordering every single pixel, we would assign every pixel an index value (effectively acting as its sequence number, mirroring the logic we already used in the pixel scheduler), and we would write the pixel directly to DDR where it would be placed correctly according to this index value. Not only did this reduce our LUT usage by 37.2%, but also

increased the value of our worst negative slack by 1.615 ns, indicating to us that we could make our design much more efficient in later versions.

5.4.2 Iteration Core Timing Violations

The main challenge we faced with our iteration core block was the trade-off between maintaining an extensively parallelised and ensuring timing requirements were met. In previous designs, we had attempted to pass a significant amount of data through combinational logic, which ended up being the critical path.

Summary						
Name	Path 1					
Slack	-1.269ns					
Source	main_v1_i/iter_core_array_0/inst/core_gen[10].single_core/ref_orbit_ram_reg_0/CLKBWRCLK (rising edge-triggered cell RAMB36E1 clocked by clk_fpga_0 (rise@0.000ns fall@6.667ns period=13.333ns))					
Destination	main_v1_i/iter_core_array_0/inst/core_gen[10].single_core/s2_p3_r_reg[17]_0/D (rising edge-triggered cell FDRE clocked by clk_fpga_0 (rise@0.000ns fall@6.667ns period=13.333ns))					
Path Group	clk_fpga_0					
Path Type	Setup (Max at Slow Process Corner)					
Requirement	13.333ns (clk_fpga_0 rise@13.333ns - clk_fpga_0 rise@0.000ns)					
Data Path Delay	14.291ns (logic 6.785ns (47.478%) route 7.506ns (52.522%))					
Logic Levels	16 (CARRY4=10 LUT2=1 LUT3=1 LUT6=4)					
Clock Path Skew	-0.170ns					
Clock U...ainty	0.203ns					
Source Clock Path						
Delay Type	Incr (ns)	Path (ns)	Location	Cell Pin	Cell	Netlist Resources
(clock clk_fpga_0 rise edge)	(r) 0.000	0.000				
PS7	(r) 0.000	0.000	Site: PS7_X0Y0	FCLKCLK[0]	PS7_i (PS7)	main_v1_i/processing_system7_0/inst/PS7_i/FCLKCLK[0] main_v1_i/processing_system7_0/inst/FCLK_CLK_unbuffered[0]
BUFG (Prop. bufg_1_O)	(r) 0.101	1.294	Site: BUF...TRL_X0Y16	O	buffer_fclk_clk_0.FCLK_CLK_0_BUFG (BUFG)	main_v1_i/processing_system7_0/inst/buffer_fclk_clk_0.FCLK_CLK_0_BUFG/O
net (fo=34142, routed)	1.795	3.089				main_v1_i/iter_core_array_0/inst/core_gen[10].single_core/clk main_v1_i/iter_core_array_0/inst/core_gen[10].single_core/ref_orbit_ram_reg_0/CLKBWRCLK
RAMB36E1			Site: RAMB36_X1Y1	CLKBWRCLK	ref_orbit_ram_reg_0 (RAMB36E1)	
Data Path						
Delay Type	Incr (ns)	Path (ns)	Location	Cell Pin	Cell	Netlist Resources
RAMB36E1 (Prop. ramb36_CLKBWRCLK_DOBDO[30])	(r) 2.454	5.543	Site: RAMB36_X1Y1	DOBDO[30]	ref_orbit_ram_reg_0 (RAMB36E1)	main_v1_i/iter_core_array_0/inst/core_gen[10].single_core/ref_orbit_ram_reg_0/D
net (fo=2, routed)		1.038	6.582			main_v1_i/iter_core_array_0/inst/core_gen[10].single_core/s1_z_ref_r[4]
LUT6 (Prop. lut6_i0_O)	(r) 0.124	6.706	Site: SLICE_X20Y8	O	s2_p3_r0_0_i_32_9 (LUT6)	main_v1_i/iter_core_array_0/inst/core_gen[10].single_core/s2_p3_r0_0_i_32_9/O
net (fo=1, routed)		0.000	6.706			main_v1_i/iter_core_array_0/inst/core_gen[10].single_core/s2_p3_r0_0_i_32_9_n
CARRY4 (Prop. carry4_S[11] CO[3])	(r) 0.533	7.239	Site: SLICE_X20Y8	CO[3]	s2_p3_r0_0_i_20_9 (CARRY4)	main_v1_i/iter_core_array_0/inst/core_gen[10].single_core/s2_p3_r0_0_i_20_9/CC

Figure IX: Failed timing report for iteration core

In Figure IX we can see the timing report of the critical path, which was one of the many timing violations we ran into. In our static timing analysis, we noticed that this was a setup violation mainly caused by two things: First, in the table named Logic Levels, we can see that this path requires 16 logic levels. This is an extremely long and complex path especially complex for a clock period of 13.333 ns (75 MHz).

Secondly, the Net delay, seen in the table labelled net in the Data Path box, is relatively large at 1.038 ns. This suggests that there is congestion and routing delay which is causing timing to fail as the signal must travel too far, or to too many destinations.

In order to fix both of these issues, we decided to increase the number of pipelined stages to seven. This ensured that complex combinational logic was not done all over one cycle, but was rather split across multiple cycles thus allowing us to pass timing in our final releases. This is key as when zooming into the fractal sets, signals not reaching their respective destinations in time would become extremely apparent in both pixelated and incorrect images being displayed.

5.5 Version Releases

We made releases whenever there were large changes and updates to the architecture. This allowed us to keep track of the progress we made, and document the important design decisions we made.

5.5.1 Version 1

Version 1 was the first working design which managed to display a fractal on screen. It used our old architecture with the pixel scheduler, reorder buffer, and packer. While it worked, there were major bugs, and it would often fail to render at higher zooms, and reset behaviour was ineffective for continuous streaming.

5.5.2 Version 1.1

Firstly, we made changes to the reorder buffer, increasing its size to 8192 rather than 4096. This fixed an issue we found where the reorder buffer would fill to its maximum size which would cause the render to fail.

A few other small changes to reset logic within the block design were made, which fixed an issue where rendering broke after loading one fractal.

Our baseline of a Mandelbrot set gave a total end-to-end latency of 1631.5250 ms. This design behaved more like a one-shot renderer than a continuous HDMI stream, so we needed to make a few improvements for our final design.

5.5.3 Version 2

We created our pixel write engine, which also replicated the functionality of the pixel packer given to us in the project brief repository.

On the PS side, we also implemented double-buffering, so that the PL side would write into one buffer while the other buffer would write into HDMI. This stopped screen tearing, leading to a cleaner frame update.

5.5.4 Version 2.1

The colour palette was updated to use .mem files in BRAM, along with logic to scale the palette based on the max iteration count. Since palettes had 1024 entries, a non-scaled palette's colour would be concentrated at one end. In this way, we saw more variety and therefore got a more interesting image.

We then replaced the pixel scheduler with a tile scheduler. This paved the way for progressive rendering and adding dirty rectangles to skip rendering where we knew the UI was going to be.

5.5.5 Version 3

The final version was focused on implementing further extensions to maximise our throughput. We also added the ability to cycle through colour palettes. For throughput maximisation we included extensions such as periodicity checking, and the Mariani-Silver Algorithm.

5.6 Hardware Extensions

In our v2.1 and v3 releases, we focused on challenging ourselves to implement complex extensions to obtain the largest throughput speedup multiple possible, some of these extensions were successful, but others were unable to be implemented.

5.6.1 Dirty Rectangles

The dirty rectangles feature was implemented in the tile scheduler which ensured that we would not calculate values in the top left and right corners as our user interface would overlay these pixels. This reduced the amount of computation required in our overall design but didn't affect latency significantly given that these rectangles were in regions with very quick escape iterations.

5.6.2 Progressive Rendering

The idea of progressive rendering was to produce an image of a lower resolution with extremely low latency which updates over time becoming clearer. This would provide a lower user-perceived latency. In Figure X we see the total and perceived latency of this implementation in our v2 release. This gave us a perceived latency speedup multiple of ~ 5.73 relative to our v1.1 design.

```
Total Progressive Render Time: 0.630s
Mandelbrot v2 render: PASS
The end-to-end render time for Mandelbrot v2 is 0.06099557876586914 seconds
```

Figure X: Timing data for the version 2 release

5.6.3 Periodicity Checking

One of our successful v3 implementations was periodicity checking. This implementation allowed us to return an iteration value, and hence a corresponding pixel colour, in the cases where values in the sequence start to repeat each-other. Rather than completing every iteration, we would exit this loop earlier allowing for a significant drop in cycle calculation which significantly increased our throughput multiple relative to the CPU baseline when the maximum number of iterations was increased.

5.6.4 Mariani-Silver Algorithm

Another successful v3 extension was the addition of the Mariani-Silver algorithm, an algorithm which determines the colour of pixels within the perimeter of a rectangle. If these pixels are uniform in escape iteration, the algorithm fills interior pixels with this corresponding colour, avoiding individual pixel iteration. This was deemed to be significant in extremely stable regions as not only was the colour representing these pixels calculated faster due to periodicity checking, but the number of slower calculations was also reduced as only a fraction of them

were calculated. When maximising our core usage, we achieved a total render time of 84ms. This is an 87% improvement on latency relative to version 2.1.

```
Elapsed: 0.073s
Pixel write rate: 11.048 Mpixels/s

Total Progressive Render Time: 0.084s
Mandelbrot v2 render: PASS
The render time for Mandelbrot v2 is [0.08384275436401367] seconds
```

Figure XI: Timing data for the version 3 release

5.6.5 Scaled Perturbation

One unsuccessful extension we attempted was the application of perturbation theory (see Appendix C) to allow us to increase our zoom factor giving us a static "deep-zoom" image. This was meant to overcome the zoom limitations of the Q4.22 number format through a combination of high precision mathematics in python, with heavy computational logic in System-Verilog.

Ultimately, this implementation was unsuccessful due to fundamental dynamic range limitations in the Iteration Core block's escape logic. Standard pixels escape when their mathematical orbit crosses a radius of 2.0. However, because our hardware was evaluating a scaled variable, the required hardware escape condition effectively became 2.0 multiplied by a scale factor $\gg 1$.

Our 26-bit Q4.22 format has a hard physical maximum of 7.99. Hence, the scaled orbits instantly overflowed the hardware limit within the first few clock cycles, triggering false escapes and rendering the tiles as uniform solid colours. Resolving this would require widening the fundamental fixed-point datapath to over 60 bits to satisfy the dynamic range, an extension we deemed architecturally unfeasible with the DSP budget.

5.7 Testing

Testing was split into two stages: RTL simulation and PS tests on the PYNQ-Z1. Testing each layer separately allows us to isolate faults before attempting full integration, so whenever we would make updates to individual modules, we would testbench first.

5.7.1 RTL Simulation & Verification

Individual SystemVerilog testbenches were written for the main hardware blocks before they were integrated into the full Vivado block design. The purpose of these simulations was not only to check functional correctness, but also to verify that metadata such as pixel indices, mode values, and valid/ready handshakes were preserved correctly through the pipeline.

The tile scheduler was tested using a reduced-resolution frame so that tile traversal could be inspected quickly in simulation. This allowed us to check that the scheduler generated the expected tile order, emitted correct pixel indices, and produced the correct complex-plane coordinates from the supplied fixed-point step sizes. Testing at a smaller resolution made the simulation easier to debug while still exercising the same scheduling logic used in the full 1280x720 design.

The iteration core and iteration core array were tested to confirm that each core accepted work through the valid/ready interface and returned the correct escape information and iteration count. These tests also verified that the pixel index attached to each input was preserved when the result returned, which is essential in the final direct-framebuffer architecture because pixels can complete out of order.

The colour palette was tested to confirm that iteration counts were converted into RGB values while preserving the corresponding pixel index. The final palette design also required additional checks because it supports runtime palette selection, iteration-count scaling, and tile-fill inputs from the Mariani-Silver path. Visual testing later confirmed that the palette selection and scaling behaved as expected on the HDMI output.

The Pixel Write Engine was one of the most important blocks to verify because it replaced the older reorder-buffer and ordered streaming path. Its testbench supplied unordered pixel indices and checked that each pixel was written to the correct framebuffer address. The testbench also exercised AXI write-response error counting, enable gating, and soft-reset behaviour. This gave confidence that the writer could accept out-of-order coloured pixels and use their pixel indices to reconstruct the correct framebuffer layout in DDR.

5.7.2 End-to-end System Testing

After unit-level simulation, the blocks were integrated into the Vivado block design and tested on the PYNQ-Z1. The main aim at this stage was to confirm that the full PL pipeline could write a valid 720p framebuffer into DDR and that the HDMI path could display it correctly.

The hardware tests focused on observable outputs and internal status counters. The PS programmed the tile scheduler registers with the current view parameters, selected the fractal mode and maximum iteration count, set the framebuffer address for the Pixel Write Engine, and then started a render pass. During rendering, the PS monitored the writer status counters, including accepted pixels, completed writes, and write errors. This helped distinguish between computation errors, AXI write failures, and display-path issues.

5.7.3 Processing-System Testing

The processing system was also used as a diagnostic tool throughout the project, especially during the early stages of hardware integration. Rather than immediately building the final application, we first used Jupyter notebooks on the PYNQ-Z1 to interact directly with the hardware design. This allowed us to load bitstreams, inspect the hardware handoff file, discover memory-mapped IP blocks, write AXI-Lite registers, allocate framebuffers, and check status registers without needing the full user interface or controller stack.

Jupyter notebooks were also used to compare hardware output against software-rendered reference images. The PS could generate or display a reference fractal using Python, then run the PL renderer with the same parameters and compare the result visually. This was useful for identifying coordinate-conversion errors, RGB byte-order mismatches, palette scaling issues, and mode-specific problems.

6 Custom Controller

6.1 System Requirements

Before designing the hardware architecture, a strict set of top-level requirements was established to ensure that the controller would function as a reliable, low-latency input device for the FPGA processing system.

The key requirements were:

- The system must capture physical inputs with less than 10 ms of end-to-end latency to ensure fluid user interaction. To achieve this, hardware polling must run at a minimum of 1 kHz, with state updates transmitted at a continuous 100 Hz.
- The communication protocol must guarantee data reliability over UART at a baud rate of 115200. The system must be capable of detecting and dropping corrupted frames caused by EMI or signal degradation before they reach the processing pipeline.
- The hardware must support simultaneous multi-input processing without ghosting or blocking.
- The controller must be powered entirely via a standard 5V USB connection, drawing under 100 mA at peak operation.

6.2 Breadboard Prototyping

Before committing to a custom PCB layout in KiCAD, the entire circuit was prototyped on a breadboard using a Raspberry Pi Pico as a breakout for the RP2040 microcontroller. This intermediate step was critical for validating the electrical characteristics of the input peripherals and informing the firmware design.

The breadboard phase highlighted several physical constraints that directly influenced the final implementation:

- Initial tests connecting the tactile buttons to the RP2040's GPIO pins revealed significant contact bounce upon both actuation and release. Using an oscilloscope, we measured the bounce duration, which occasionally exceeded 2 to 3 milliseconds. This empirical data justified the design decision to implement a strict 5-millisecond vertical-counter software debounce in the final firmware, ensuring clean edge detection without relying on bulky external RC hardware filters.
- Prototyping the rotary encoders revealed that simple polling loops routinely missed steps during rapid, coarse rotation. This observation proved that the RP2040's hardware interrupts (ISRs) were strictly necessary for the encoder pins. Testing on the breadboard allowed us to fine-tune the 16-entry state-machine lookup table, ensuring deterministic clockwise and anti-clockwise tracking even when the encoder was spun aggressively.
- The analog joystick was wired to the RP2040's internal 12-bit ADCs. By streaming the raw analog values to a serial monitor while the joystick was idle, we observed a natural voltage fluctuation. This baseline noise profile was crucial; it provided the quantitative data required to define the mathematical software deadzones and calculate the dynamic center-offset logic used in the final boot sequence.

6.2.1 Design Process

Highlighting the design process, an early iteration of the circuit relied on two linear potentiometers for fine parameter control rather than utilising the full digital functionality of rotary encoders. Ergonomically, the potentiometers resulted in a clunkier user experience; they were bound by physical rotational limits and lacked the infinite adjustability and dynamic software scaling that encoders provide. More significantly, this design introduced major hardware bottlenecks. The combination of the potentiometers and the dual-axis joystick required four analogue inputs in total. Because the RP2040 microcontroller exposes only three external ADC pins, this architecture forced the inclusion of an external analogue multiplexer to cycle through the sensor readings. Pivoting to the final digital encoder design was a critical optimization. By eliminating the multiplexer, we drastically reduced PCB routing complexity, lowered component count (BOM), and removed the analogue crosstalk, switching latency inherent to multiplexed signals. This allowed the ADCs to be dedicated entirely to clean, high-speed sampling of the joystick, while shifting the parameter controls to the RP2040's abundant digital GPIO pins. This was one of the major changes we made from our initial idea, which demonstrated to us the importance of prototyping and iterative design.

6.3 Hardware Design

6.3.1 PCB Design

The controller PCB was designed using KiCAD. To ensure electrical reliability, a hybrid routing strategy was used. Critical power tracks were manually routed with increased track widths to handle current demands safely.

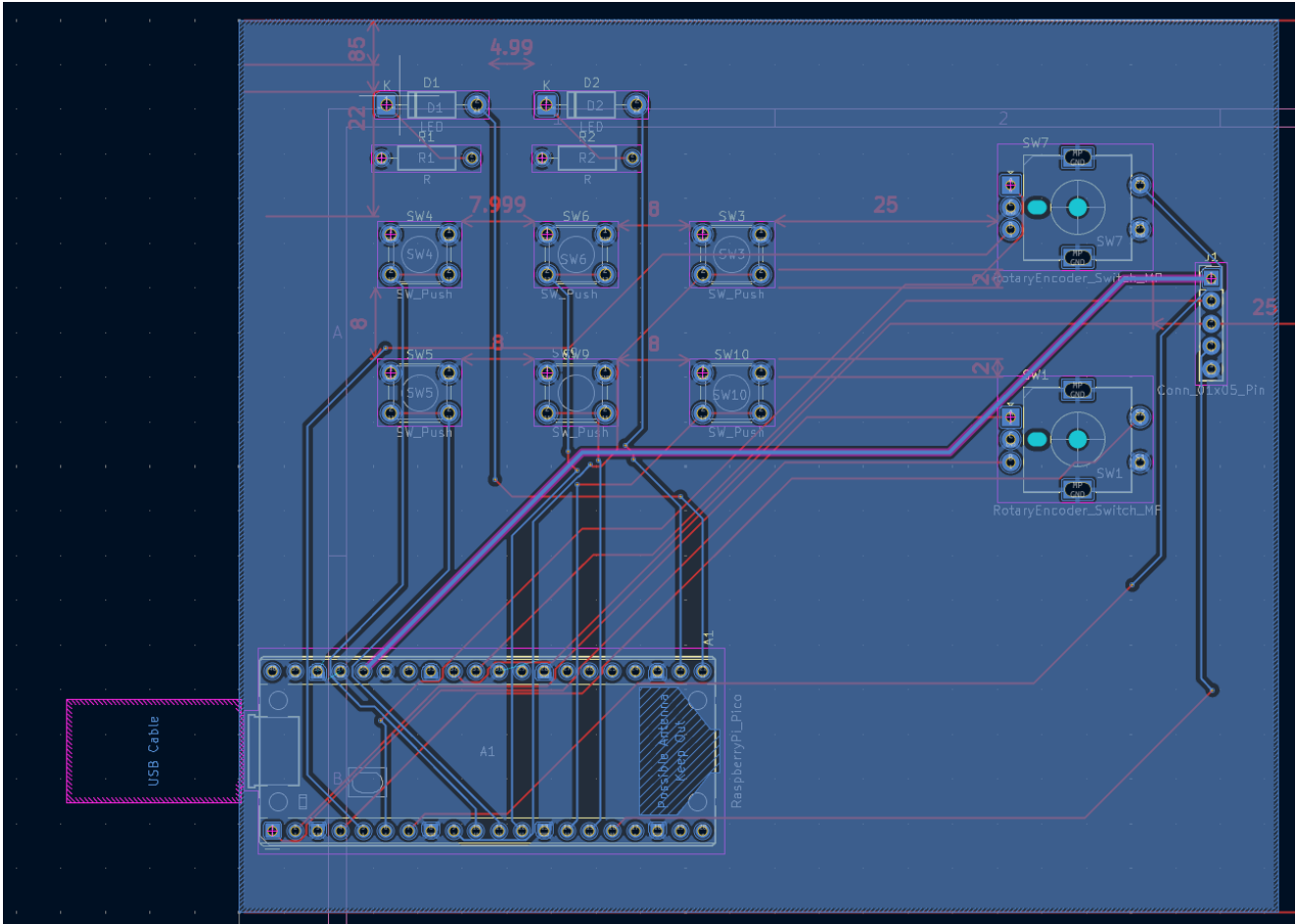


Figure XII: Controller PCB diagram

In addition, a continuous copper ground pour was used across the bottom layer. Once power and ground were finalised, the remaining logic tracks were routed using an automated logical router to optimise track paths and save time.

6.4 Firmware Design

The firmware for the RP2040 was developed in C to adhere to the system’s architecture. The firmware’s final responsibility is to sample the hardware, package the input states, and transmit them to the processing system. Hardware polling and filtering are scheduled at 1kHz, while packet construction and USB CDC serial link transmission operate at 100 Hz. This ensures the physical inputs are captured with zero perceivable latency while also not overwhelming the processing system.

6.4.1 Buttons

A vertical-counter debounce algorithm was implemented to handle the tactile buttons, encoder switches, and joystick switch. The firmware maintains a continuous 1kHz poll of all active-low GPIO pins. For each button, the system tracks consecutive polls that disagree with the currently recorded state. A pin has to register a consistent state change for 5 consecutive milliseconds for the new state to be accepted and updated. This filters out mechanical contact bounce on both the press and release edges. Furthermore, edge-detection logic is applied to isolate the exact frame a button is pressed, ensuring that state-toggling commands only trigger once per physical click.

6.4.2 Encoders

To read the rotary encoders, it was imperative that steps would not be missed during rapid rotation. This was achieved using an Interrupt Service Routine triggered by edge-detects on the encoder GPIO pins. To decode the rotational direction and reject mechanical noise, the ISR utilises a state-machine approach driven by a 16-entry lookup table. By shifting the previous 2-bit state of the A/B channels and combining it with the current 2-bit state, the firmware creates a 4-bit index. This index is passed through the lookup table, which returns +1 for a

clockwise step and -1 for an anticlockwise step. To construct the packet, the main loop divides by 4 (to account for the detents) and applies dynamic software multipliers that allow the user to toggle between 'fine' and 'coarse' adjustment using the encoder switches.

6.4.3 Joystick

The analogue joystick positions are read via the RP2040's internal ADCs. An auto-calibration routine was built into the boot sequence. On power-on, the firmware delays for a short settling period before taking 50 rapid samples of both the X and Y axes and using the final settled sample as the baseline centre offset. The firmware subtracts this dynamic centre from the raw ADC readings to provide a true zero-centred output. To prevent cursor drift caused by noise, a software deadzone is applied to the values.

6.4.4 Data Integrity

To guarantee data integrity between the controller and the processing system, a robust serial communication protocol was implemented. The firmware constructs a structured ASCII packet containing a fixed header (TDT), the raw hardware values, and the button bitmask. Before transmission over USB CDC serial link, the firmware calculates a CRC8-CCITT checksum of the payload and appends it to the end of the packet. If electrical noise or signal degradation corrupts the packet during transmission, the processing system's driver recalculates the checksum, detects the mismatch, and drops the corrupted frame, preventing erratic navigation or system crashes.

6.5 Testing & Verification Results

Before attempting integration between the controller and the FPGA, we decided to outline a series of minor tests to ensure the stability and validity of the controller and its output. These tests fell into 5 main categories- these were:

- Power-on and USB Enumeration
- CRC Integrity
- Encoder Testing
- Button Testing
- Joystick Testing

The following verification matrix details the testing procedures and pass/fail criteria for the core hardware subsystems, including power-on behaviour, data integrity, and peripheral inputs.

Subsystem	Test Target	Observed Result / Verification Metric	Status
Power-on & USB Enumeration	Green LED Behaviour	Blinks consistently at 1 Hz	Pass
	Red LED Behaviour	Stays completely off at idle state	Pass
	Packet Stream Rate	Packets stream continuously at 100 Hz	Pass
	Packet Formatting	Format strictly matches protocol specifications	Pass
CRC Integrity	Python Validation	CRC8-CCITT checksum validated; 0 failures over 50+ packets	Pass
	Sequence Tracking	Numbers increment without gaps over 30-second window	Pass
Encoder Testing	ZOOM Direction (Enc0)	CW rotation yields positive <code>zoom_d</code> ; CCW yields negative	Pass
	ZOOM Accuracy (Enc0)	100-click test achieved exact +100 (CW) and -100 (CCW) targets	Pass
	ITER Core (Enc1)	Identical sign convention and accuracy metrics verified	Pass
	Idle Stability	Zero phantom counts recorded at idle state	Pass
Button Testing	Bitmask Mapping	Each button triggers the correct assigned bitmask bit	Pass
	Debounce Stability	No chatter on press/release; bits transition cleanly once	Pass
Joystick Input	Center Calibration	Centered position consistently reads near (0, 0)	Pass
	Full Deflection	Reaches expected maximum range of $\pm \sim 1968$	Pass

Table 8: Hardware Subsystem Verification Matrix

Button bit map reference:

Bit	Signal Name	Physical Control	GPIO	Expected btn_hex	Verification Status
0	NEXT	Button 2	GP10	0001	Passed
1	BACK	Button 1	GP9	0002	Passed
2	ACTION	Button 3	GP19	0004	Passed
3	FUNCTION	Button 4	GP20	0008	Passed
4	PALETTE	Button 5	GP12	0010	Passed
5	MENU	Button 6	GP15	0020	Passed
8	RESET	Joystick switch	GP12	0100	Passed

Table 9: Final Controller Button Mapping and Hexadecimal Values

6.6 Case Design

The last stage of producing a physical controller was to design a 3D model of a casing that would hold the PCB design securely and allow access to all of the inputs before getting a 3D print produced and making any revisions to the design if necessary.

For this process, we used Kicad in order to produce a STEP file of the PCB in order to import a 3D render of the PCB into FreeCAD, where we would design the casing to scale to match the render in order to reduce the likelihood of any issues with tolerance or dimension mismatching.

In FreeCAD, two separate bodies were designed- a lid and a tray. Both of which had their complications due to certain limitations imposed by trying to keep the designs mechanically simple as to reduce complexity.

The tray design was the simpler of the two, only requiring a few features in order to work appropriately with the PCB design, these were:

- Standoffs - Necessary for the PCB to sit on to allow the inputs to be flush with the lid (whilst also avoiding the pins of any components)

- Lip - Allows the lid to latch onto the tray smoothly for easier assembly
- Screw Platforms - Points where the lid can screw into the tray

Below is an image from FreeCAD showcasing the tray design:

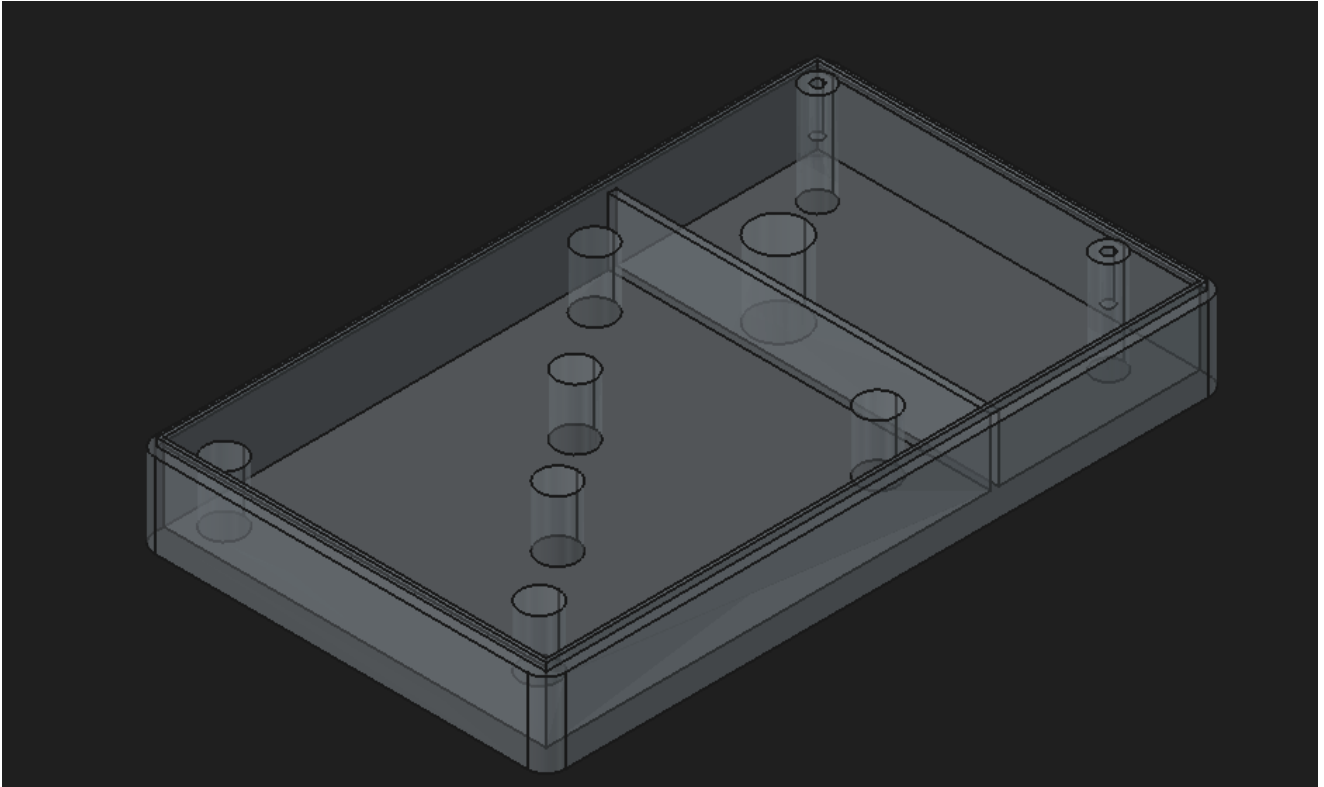


Figure XIII: Tray 3D Model

The use of standoffs and a partition wall specifically help reduce complexity by mitigating the need for any physical assembly of the PCB to the casing- meaning it can be easily taken apart at any time and massively simplifies the design of both the PCB and tray.

Similarly, the lid had its own requirements for it's design- alongside other limitations we had to work around when designing.

- Height limitations - Each component present has a different height (ENC0, ENC1 sit higher than the buttons), therefore, multiple levels had to be used.
- USB Connection - The USB connection point sits higher than the push switches, meaning it had to be incorporated into the lid.
- Assembly - Screw holes were added to allow quick and secure assembly.
- Input Accessibility - As mentioned before, each input interface had a different height level, which would need to be accounted for.

Below is an image from FreeCAD showcasing the lid design and the full design together with the PCB model:

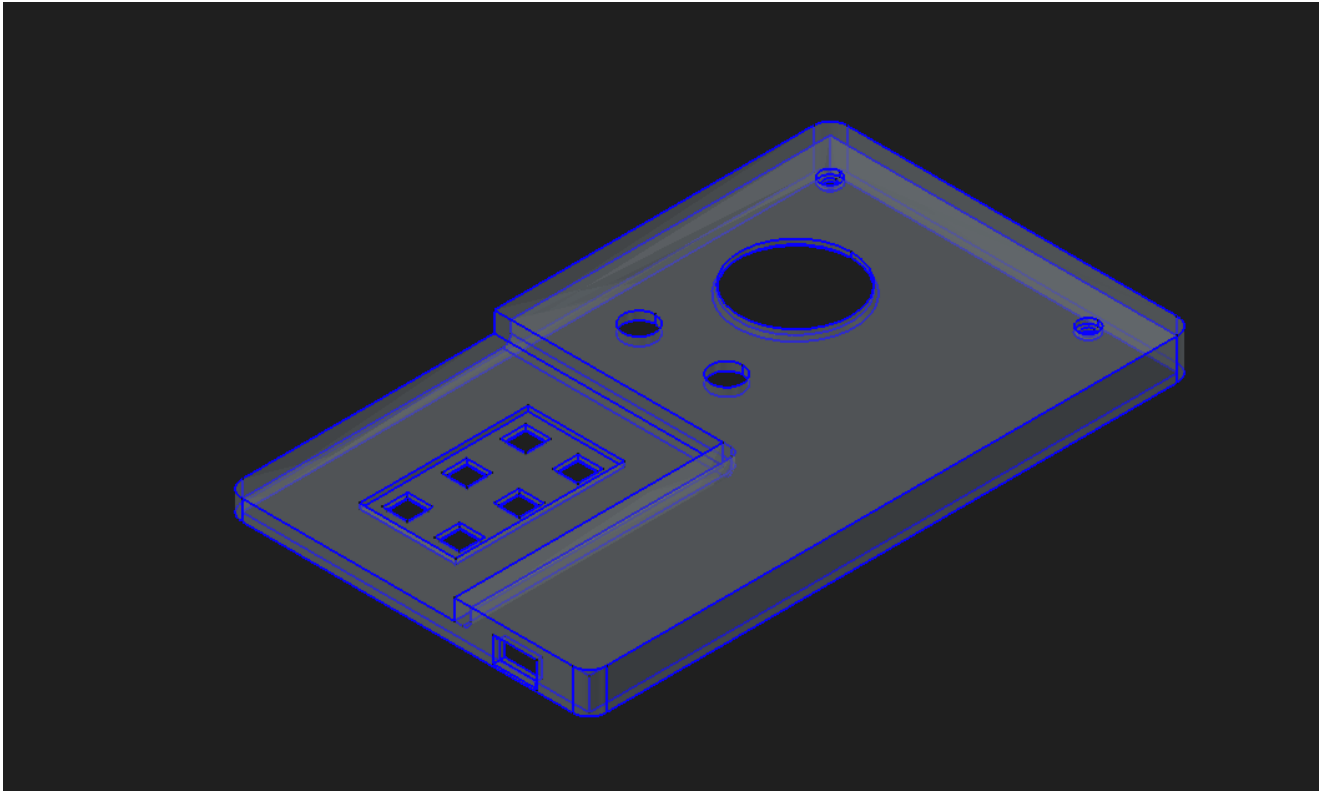


Figure XIV: Lid 3D Model

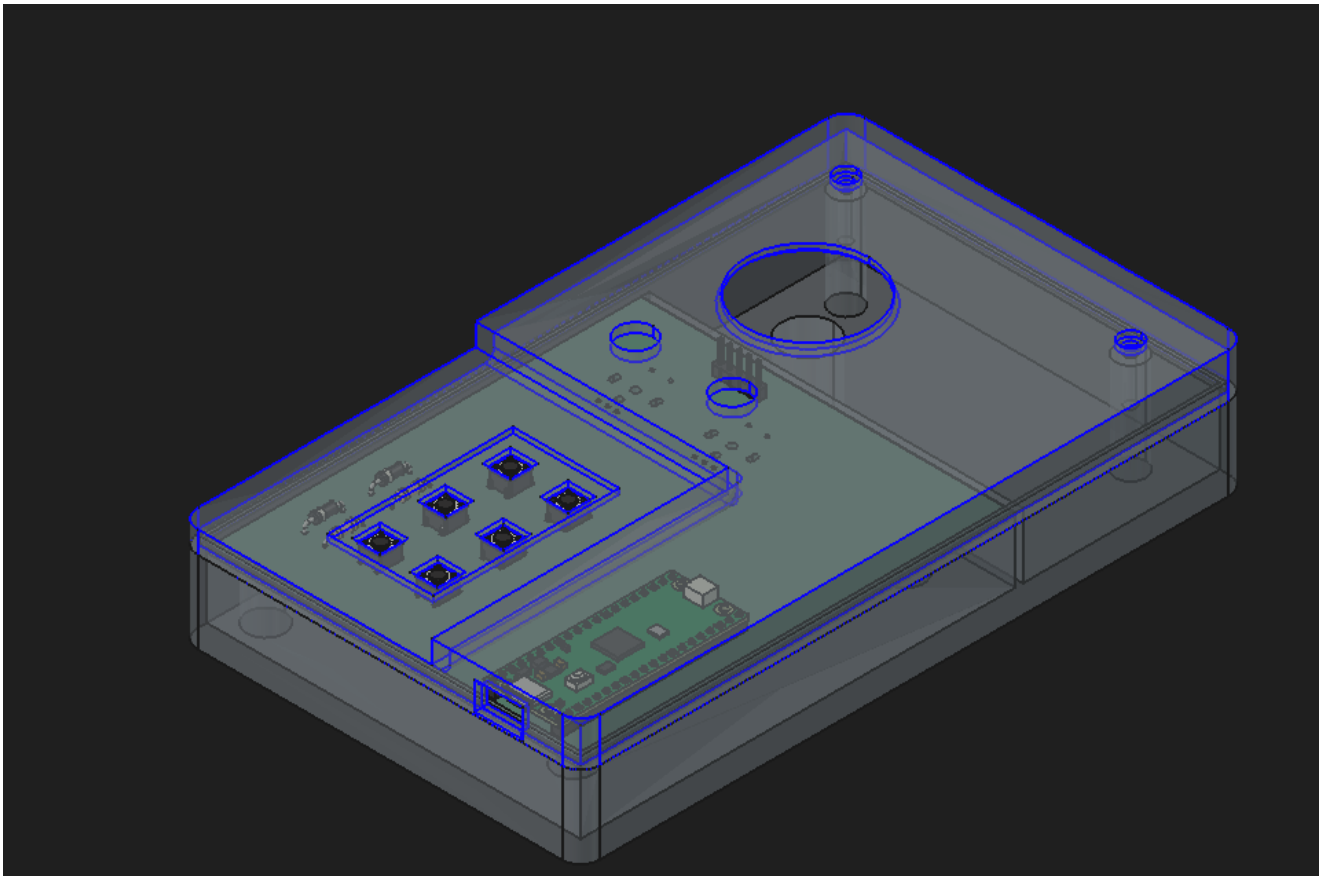


Figure XV: Full 3D Model

As shown above, the model houses the PCB as intended whilst still allowing for small tolerances to allow for minor discrepancy during the printing process that could hinder a number of different things such as the PCB sitting too high causing the lid to not sit flush for proper assembly.

7 Processing System

7.1 Role of the PS

The PS is the control layer of FractalScope. It connects the Pico USB controller or keyboard fallback to the PL, maintains application state, programs the FPGA registers, manages framebuffers, composites the user interface, and drives the HDMI display path. The PL is therefore kept focused on high-throughput fractal computation, while the PS handles the parts that change frequently during development: scene flow, controls, educational text, menus, and benchmarking.

For photos of the Actual PS, refer to Appendix E

At runtime, the PS performs four main jobs. First, it converts controller or keyboard input into shared application events such as PAN, ZOOM, ITER, PALETTE, NEXT, and BACK. Secondly, it updates the active scene state, including the current centre coordinate, view width, maximum iteration count, selected fractal mode, Julia parameter, palette index, and HUD visibility. Thirdly, for PL-backed scenes, it converts those values into hardware-friendly register writes. Finally, it commits completed PS-rendered or PL-rendered frames to the HDMI output, applying the PS overlay pass where required.

This split made the design easier to develop because changes to the educational walkthrough, menu, controller mapping, or HUD layout did not require a new bitstream. It also gave the hardware a compact interface: the FPGA receives view parameters, a mode value, a palette control value, a framebuffer address, and a render-scale command, rather than needing to understand scene logic or text rendering.

7.2 Software Architecture

The current PS application is organised around `run_demo.py`, which acts as the integrated scene manager. It loads the individual scene modules, owns the input polling loop, routes events to the active scene, and shares one PL/HDMI backend across all hardware-rendered modes. The main software components are summarised in Table 10.

Component	Current responsibility
<code>run_demo.py</code>	Integrated application shell: input polling, scene adapters, transitions, redraw scheduling, and menu routing.
<code>scenes/scene1.py-scene4.py</code>	PS-rendered educational scenes introducing the recurrence, complex plane, escape radius, and colouring.
<code>scenes/scene5.py</code>	Guided Mandelbrot scene: intro slides plus PL-rendered Mandelbrot exploration.
<code>scenes/scene6.py</code>	Guided Mandelbrot–Julia scene with a Mandelbrot reference map and PL-rendered Julia output.
<code>scenes/summary.py</code> and <code>scenes/menu.py</code>	PS-rendered walkthrough ending, global menu, help page, and launch interface for free-roam modes.
<code>backend/pl_backend.py</code>	Shared PL backend: overlay loading, MMIO discovery, VDMA setup, PYNQ buffer allocation, scheduler/writer programming, palette control, render polling, and HDMI commits.
<code>backend/free_roam_fractals.py</code>	Menu-launched free-roam Mandelbrot, Julia, Burning Ship, and Tricorn state handling and HUD drawing.
<code>backend/cpu_baseline_runner.py</code> and <code>cpu_baseline/</code>	C++ CPU renderer build/run wrapper, raw RGB loading, resizing, packing, and CPU-vs-PL comparison support.

Table 10: Current PS-side software components.

The guided PL scenes and the free-roam scenes are intentionally separate. Scene 5 and Scene 6 are controlled demonstrations used inside the walkthrough, whereas the menu launches independent free-roam adaptors with their own state. This prevents tutorial state, menu state, and exploration state from being mixed together after the walkthrough has finished.

7.3 Input Abstraction and Controller Handling

The PS supports both the custom Pico USB controller and a keyboard fallback. Both are converted into the same `AppEvent` representation, so the scene code does not need separate keyboard and controller paths. This was useful during development because the whole application could be tested over SSH before the physical controller was connected.

The Pico appears to the PYNQ board as a USB CDC serial device, normally under `/dev/ttyACM*`. The PS opens it as a non-blocking character device and automatically falls back to keyboard-only operation if the controller is missing or disconnected. The current parser accepts controller packets of the form

`TDT,<seq>,<btn_hex>,<zoom_d>,<iter_d>,<jx>,<jy>,<crc>`

where `seq` is a sequence counter, `btn_hex` is the button bitmask, `zoom_d` and `iter_d` are signed encoder deltas, `jx/jy` are joystick readings, and `crc` is a CRC8-CCITT checksum. The PS recomputes the checksum before accepting a packet. Malformed packets are ignored rather than blocking the application loop, and controller disconnection simply returns control to the keyboard fallback.

Only edge-triggered button presses are converted into events, while joystick pan events repeat at a controlled interval when the stick remains outside the deadzone. This avoids flooding the scene manager with repeated button events but still gives smooth panning. The resulting event set is shown in Table 11.

Event group	Examples
Navigation	NEXT, BACK, MENU, QUIT
View control	PAN, ZOOM, RESET
Rendering control	ITER, PALETTE
Scene-specific control	ACTION, FUNCTION

Table 11: Shared application events generated from keyboard or controller input.

7.4 Scene Management and Application State

The scene manager turns separate scene scripts into one continuous application. Each scene is wrapped by an adapter with a common interface: `on_enter()`, `handle_app_event()`, `update()`, `wants_redraw()`, `draw_packed()`, and `clear_transition()`. This lets PS-rendered scenes, PL-rendered scenes, the menu, the summary page, free-roam scenes, and the CPU comparison mode be scheduled in the same loop.

PS-rendered scenes use dirty flags: they redraw only when their state changes or when an animation is still active. PL-rendered scenes work differently. Their `update()` method starts hardware renders directly through the shared backend, so they normally return `False` from `wants_redraw()`. This avoids sending stale placeholder frames through the normal PS redraw path while a PL render is being handled.

State is kept local to each scene. Educational scenes store slide indices, animation state, palettes, and redraw flags. PL fractal scenes store the current mathematical view: centre coordinate, width, maximum iteration count, mode, Julia parameter, palette index, HUD visibility, render counters, and recent status values. Menu and summary scenes only store navigation state. This separation reduces coupling and made it easier to add free-roam modes without disturbing the guided walkthrough.

The menu is treated as the endpoint of the guided walkthrough. It launches separate free-roam versions of Mandelbrot, Julia, Burning Ship, and Tricorn, plus the CPU-vs-hardware comparison. It does not route back into the guided Scene 5 or Scene 6 implementations. For Julia free-roam, the function control toggles between exploring the Julia image and selecting the Julia parameter c on the Mandelbrot reference map.

7.5 FPGA Driver and Register Interface

The shared PL backend hides the low-level hardware interface from the scene code. Scenes update Python state objects; the backend translates those objects into MMIO writes for the tile scheduler, pixel write engine, palette control registers, and VDMA. On startup it loads the bitstream/HWH metadata, locates IP blocks by name, creates MMIO handles, allocates two PYNQ framebuffers, checks the writer geometry, and starts the VDMA memory-to-stream display path.

Completed coloured pixels are written into DDR by the PL pixel write engine, and the VDMA separately reads a 32-bit framebuffer for HDMI output.

7.5.1 View Parameter Conversion

Scenes store view parameters as floating-point values because this makes interaction simple. Before rendering, the backend computes the top-left coordinate and per-pixel increments from the centre coordinate and horizontal width. These are converted to the Q4.22 signed fixed-point format used by the PL by multiplying by 2^{22} , rounding, checking the signed 26-bit range, and writing the encoded value to the scheduler registers. Keeping this conversion in software keeps the hardware register interface simple while preserving readable scene-state variables.

The scheduler register map used by the PS is shown in Table 12.

Offset	Register	Purpose
0x00	SCH_X_JUMP	Q4.22 real-axis step per pixel.
0x04	SCH_Y_JUMP	Q4.22 imaginary-axis step per pixel.
0x08	SCH_X_MIN	Q4.22 real coordinate of the first rendered pixel.
0x0C	SCH_Y_MIN	Q4.22 imaginary coordinate of the first rendered pixel.
0x10	SCH_JUL_C_R	Q4.22 real component of the Julia parameter.
0x14	SCH_JUL_C_I	Q4.22 imaginary component of the Julia parameter.
0x18	SCH_MODE_ITER	Packed fractal mode and maximum iteration count.
0x1C	SCH_CONTROL	Scheduler run bit and render scale.

Table 12: Tile scheduler AXI-Lite register interface used by the PS.

The mode and iteration depth are packed as

$$R_{\text{mode/iter}} = (\text{mode} \ll 16) \mid \text{maxIter},$$

with mode values 0–3 corresponding to Mandelbrot, Julia, Burning Ship, and Tricorn. In Julia mode, the screen coordinate is used as z_0 and the supplied Julia parameter is used as c . In the other current modes, the screen coordinate is c and $z_0 = 0$.

7.5.2 Pixel Write Engine and Palette Control

The pixel write engine is programmed with the physical address of the render framebuffer and the expected number of writes for the current pass. It exposes accepted-pixel, completed-write, and write-error counters, which the PS polls to determine render completion. Its register interface is shown in Table 13.

Offset	Register	Purpose
0x00	WR_CONTROL	Start, enable, and soft-reset control.
0x04	WR_STATUS	Writer status flags.
0x08	WR_FRAMEBUFFER_BASE	Physical DDR address of the target framebuffer.
0x0C	WR_PIXELS_ACCEPTED	Coloured pixels accepted by the writer.
0x10	WR_PIXELS_WRITTEN	AXI writes completed.
0x14	WR_WRITE_ERRORS	Non-zero when write responses report an error.
0x18	WR_FRAME_PIXELS	Target write count for the current pass.
0x1C	WR_GEOMETRY	Encoded 1280 by 720 geometry sanity check.

Table 13: Pixel write engine AXI-Lite register interface.

Palette control is separate from the fractal scheduler. The backend computes a palette scale factor

$$S_{\text{palette}} = \left\lceil \frac{1023 \cdot 2^{16}}{\text{maxIter} - 1} \right\rceil,$$

so that the 1024-entry hardware palette remains spread across the selected iteration range. The selected palette index is also written at runtime when the palette control GPIO/register is present. The current PS names eight palettes and can cycle between them without recompiling the hardware.

A hardware render follows a strict sequence: hold the scheduler in reset, soft-reset the writer, write the expected pass count, write the render framebuffer address, start the writer, program the scheduler view and palette controls, release the scheduler with the selected scale, poll the writer counters until completion or error, then return the scheduler to reset. This ordering matters because the scheduler can begin dispatching work as soon as the run bit is set.

7.6 Framebuffer Management and HDMI Output

The framebuffer system is the boundary between PL rendering, PS overlay composition, and HDMI output. The current display format is a 32-bit packed framebuffer at 1280 by 720, so each frame occupies

$$1280 \times 720 \times 4 = 3,686,400 \text{ bytes.}$$

This is larger than a 24-bit RGB buffer but avoids awkward byte packing and gives naturally aligned PL writes and PS array operations.

The backend allocates two PYNQ framebuffers. By default, framebuffer 0 is the fixed display buffer read by the VDMA, while framebuffer 1 is used as the PL render buffer. After a render finishes, the PS invalidates the PL-written buffer if it needs to read it, expands sparse progressive passes when required, draws the HUD, flushes the buffer, copies the completed image into the fixed display buffer, and re-pins the VDMA to framebuffer 0. This fixed-display commit-copy path was chosen to avoid the tearing and left-edge wrap-around seen when the VDMA could briefly scan out the wrong frame-store address.

The VDMA is used only in MM2S mode for display. It is configured with `HSIZE=5120` bytes, `STRIDE=5120` bytes, and `VSIZE=720`. Keeping the horizontal size and stride equal to the full 32-bit row width is essential: if either value is wrong, the right side of the image can appear to wrap onto the left side of the screen.

Progressive rendering is controlled by the PS using render scales such as

$$8 \rightarrow 4 \rightarrow 2 \rightarrow 1.$$

At larger scales, the scheduler renders fewer samples. The PS then expands the sparse samples into visible blocks before committing the frame, giving a fast coarse preview. During interaction, the backend can request only a scale-8 preview; after input has been idle for a short period, queued refinement passes at scales 4, 2, and 1 sharpen the image.

The current active pixel count excludes the two fixed HUD panels at the top-left and top-right of the display. The expected writer count for a pass is therefore

$$N_{\text{writes}} = \frac{N_{\text{active}}}{s^2},$$

where s is the render scale. This prevents the PS from waiting for pixels that the PL deliberately does not generate under fully opaque HUD regions.

7.7 HUD and Overlay Composition

The HUD is drawn by the PS after the PL render has completed. This keeps text rendering, fonts, menus, prompts, controller labels, and educational annotations out of the FPGA datapath. Pillow is used for text and shape rendering, while NumPy is used for direct framebuffer manipulation and packing.

The HUD uses fixed opaque panels, currently at the top-left and top-right of the screen. These panels keep text readable over high-frequency fractal detail and create known regions that the hardware can skip. The left panel generally shows the active fractal, formula, centre, width, iteration depth, Julia parameter where applicable, and render status. The right panel shows controls, palette information, or for Julia scenes a Mandelbrot reference map with a cursor representing the selected value of c .

Different scenes use this overlay layer differently. Guided scenes use prompts and explanatory text. Mandelbrot free-roam uses the HUD for live view parameters and control reminders. Julia free-roam adds a compact Mandelbrot map, cursor, and a mode label showing whether controls are selecting c or exploring the Julia image. Because this is all PS-side, UI changes can be made quickly without changing the FPGA design.

Explicit cache management is part of the overlay path. PL-written buffers are invalidated before PS reads, and completed PS-written buffers are flushed before the VDMA displays them. This prevents stale DDR/cache contents from appearing on HDMI.

7.8 Walkthrough and Free-Roam Modes

The user-facing application is split into a guided walkthrough and menu-launched free-roam modes. The walkthrough starts with PS-rendered educational scenes covering the recurrence, complex-plane mapping, escape radius, and escape-time colouring. It then moves to PL-backed Mandelbrot and Julia demonstrations, followed by a summary screen and menu.

The free-roam modes are separate exploration environments for Mandelbrot, Julia, Burning Ship, and Tricorn. They share the same PL backend and input abstraction, but each has its own preset, formula label, default view, iteration depth, and HUD behaviour. The common control model is: joystick or WASD for panning, encoder 1 or bracket keys for zoom, encoder 2 or minus/equal keys for iteration depth, reset for default view, palette for palette cycling, action for HUD visibility, and menu/back/next to return to the menu.

Julia free-roam has one extra control mode. In image-exploration mode, pan and zoom affect the Julia viewport. In c -selection mode, directional controls move the cursor on the Mandelbrot reference map and encoder 1 changes the cursor step size. This makes the Mandelbrot–Julia relationship interactive while still using the same event abstraction as the other scenes.

7.9 CPU Baseline Integration

The CPU baseline is integrated as a menu-launched comparison scene. Its purpose is to demonstrate the value of the FPGA accelerator using the same display and scene framework, not to replace the interactive PL renderer.

The compute-heavy CPU path is written in C++. The Python wrapper compiles `render_main.cpp` and `functions.cpp` when needed, runs the renderer with command-line parameters, reads the raw RGB output and JSON timing file, resizes the result for HDMI preview, and packs it into the same 32-bit framebuffer format used elsewhere in the PS application. The supported CPU comparison sets are Mandelbrot, Julia, Burning Ship, and Tricorn. The comparison scene first renders the selected view on the CPU, briefly displays the CPU preview, then runs the equivalent PL render through the shared backend. Both paths receive matching mathematical parameters: fractal set, centre coordinate, horizontal width, maximum iteration count, and Julia parameter where applicable. The CPU preview resolution defaults to a smaller size for live responsiveness, so the comparison reports throughput in pixels per second rather than simply comparing total frame times. The scene then displays the CPU timing, PL timing, pixels written, PL render scale, and normalised throughput speedup.

8 CPU Baseline

The off-board C++ CPU implementation of fractal set calculations was designed to act as a computational baseline that our Hardware Accelerated design had to beat by a significant amount. This was done through implementing a multithreaded system to allow for fractal set computation.

8.1 Initial Design

Initially, our design consisted of a single-threaded system, which computed the escape iteration value for each pixel in a raster scan format. We would then pass an array of these iteration values to a colour palette function evaluating RGB values corresponding to the escape value, storing these in an array. We would then map the pixels and their colours to a png file through the stb image library.

The system allowed for selection of centre values and zooming factors with our four fractal sets producing accurate images.

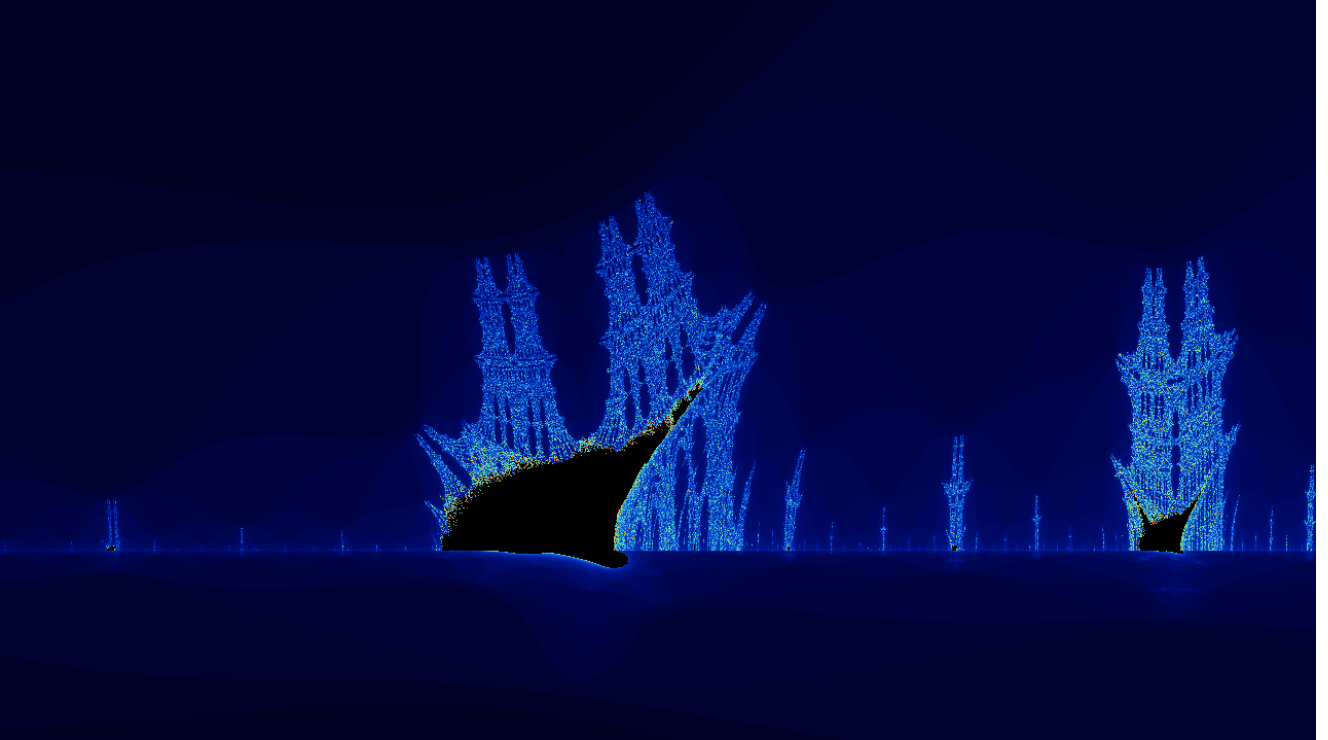


Figure XVI: CPU computed burning ship png

Given this was only single threaded, this implementation had a worst-case time complexity of $O(\text{width} \times \text{height} \times \text{max iterations})$. This was extremely inefficient and didn't provide a strong enough baseline to compare our hardware acceleration

8.2 Threaded Optimization

In order to optimise our design, we decided to implement a multi-threaded system where each thread would take a proportion of the rows of pixels that required computation, following the steps of the initial design. We decided to implement threading this way as it didn't require any synchronisation primitives. Given each row could be independently calculated, we ensured that there were no race conditions in our design.

Our off-board design ended up using 64 threads as it provided the largest speedup factor, however given that our final comparison uses PYNQ CPU cores, we were limited to using a dual-threaded system

8.3 Timing & Analysis

In order to chose the optimal number of threads for our raster scan, we ran 100 iterations of the fractal set calculation determining the average time it took for these sets to be calculated. We would select a number of threads and mapped the average time to complete the calculation, and join the threads, seen in the figures below.

Mandelbrot Timing Tests

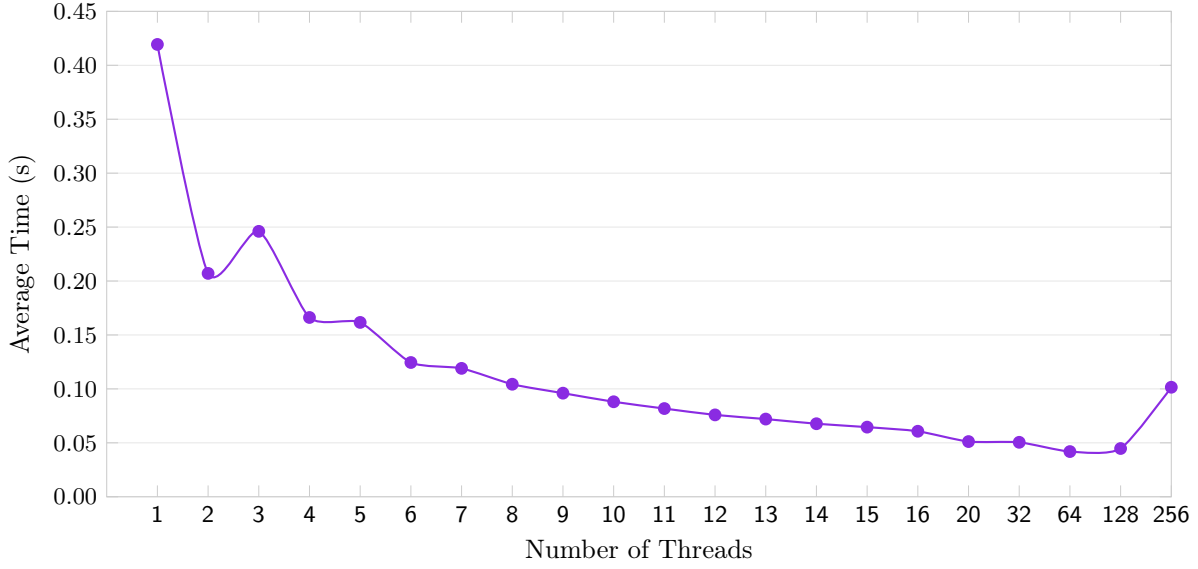
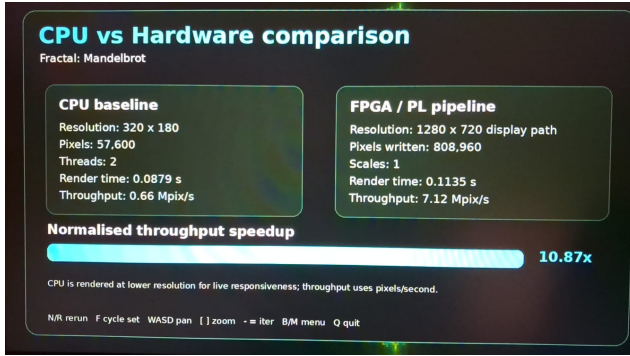


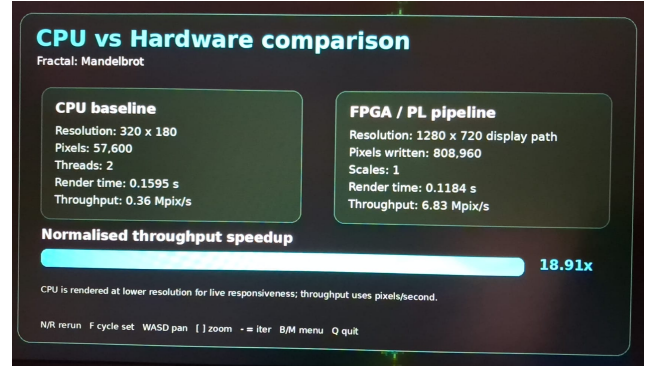
Figure XVII: Average Mandelbrot execution runtime scaling across threaded configurations.

9 Final Evaluation

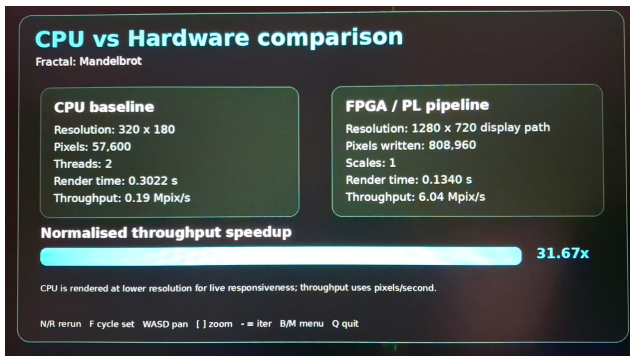
Overall, FractalScope met the main aims set out at the start of the project. The final system renders four fractal sets using a 23-core FPGA accelerator, while the processing system manages the interactive walkthrough, free-roam modes, controller input, HUD overlays, framebuffer management, and CPU comparison. The measured maximum throughput speedup of 53x, with a maximum iteration count of 2048, over the multi-threaded CPU baseline comfortably exceeds the original 10x acceleration target.



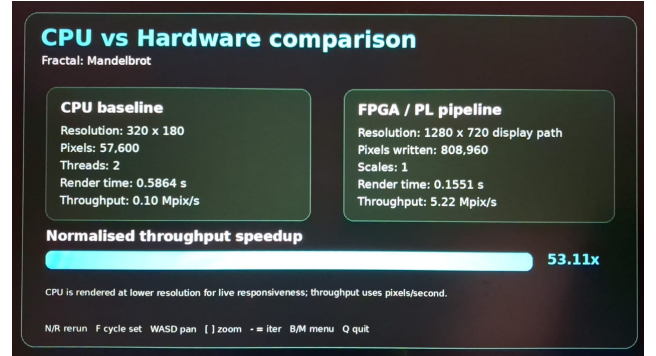
(a) 256 Maximum Iterations



(b) 512 Maximum Iterations



(c) 1024 Maximum Iterations



(d) 2048 Maximum Iterations

Figure XVIII: Throughput multiple comparisons between hardware & software

The system also met the main usability requirements. The custom controller provides intuitive physical control

over panning, zooming, iteration depth, palette selection, and menu navigation, while the keyboard fallback made the system robust during development and testing. The HUD and guided scenes support the educational goal by exposing the current fractal parameters and explaining how the mathematical recurrence maps to the generated image. The menu-launched free-roam modes then allow the user to explore the same concepts interactively. The main limitations of the final design come from deliberate architectural trade-offs. The Q4.22 fixed-point format gives efficient hardware arithmetic and allows many parallel cores to fit on the device, but it limits deep zoom precision. Some advanced acceleration ideas, such as scaled perturbation, were not practical within the available fixed-point range and DSP budget. Nevertheless, the completed design demonstrates a successful hardware/software split: the FPGA performs the high-throughput pixel computation, while the processing system provides the flexibility needed for interaction, display composition, and education.

10 Appendix

10.1 A: Set Calculations

Each screen pixel is mapped to a complex coordinate $p = x + iy \in \mathbb{C}$. The rendered colour is produced using an escape-time method. For a maximum iteration count $N_{\max}$, the recurrence is applied until either the point escapes or the maximum iteration count is reached. A point is treated as escaped when

$$|z_n| > 2,$$

which is implemented equivalently as

$$\operatorname{Re}(z_n)^2 + \operatorname{Im}(z_n)^2 > 4.$$

10.1.1 Mandelbrot Set

For the Mandelbrot set, the pixel coordinate is used as the constant c , while the initial value is fixed at zero:

$$z_0 = 0, \quad c = p.$$

The recurrence is

$$z_{n+1} = z_n^2 + c.$$

The Mandelbrot set is therefore

$$M = \{c \in \mathbb{C} : z_0 = 0, z_{n+1} = z_n^2 + c, (z_n) \text{ remains bounded}\}.$$

10.1.2 Julia Set

For the Julia set, the pixel coordinate is used as the initial value z_0 , while c is a fixed parameter selected by the user:

$$z_0 = p, \quad c = c_r + ic_i.$$

The recurrence is

$$z_{n+1} = z_n^2 + c.$$

For a chosen value of c , the corresponding filled Julia set is

$$J_c = \{z_0 \in \mathbb{C} : z_{n+1} = z_n^2 + c, (z_n) \text{ remains bounded}\}.$$

10.1.3 Burning Ship Set

For the Burning Ship set, the pixel coordinate is used as the constant c , while the initial value is fixed at zero:

$$z_0 = 0, \quad c = p.$$

The recurrence replaces the real and imaginary components of z_n with their absolute values before squaring:

$$z_{n+1} = (|\operatorname{Re}(z_n)| + i|\operatorname{Im}(z_n)|)^2 + c.$$

The Burning Ship set is therefore

$$B = \{c \in \mathbb{C} : z_0 = 0, z_{n+1} = (|\operatorname{Re}(z_n)| + i|\operatorname{Im}(z_n)|)^2 + c, (z_n) \text{ remains bounded}\}.$$

10.1.4 Tricorn Set

For the Tricorn set, also known as the Mandelbar set, the pixel coordinate is used as the constant c , while the initial value is fixed at zero:

$$z_0 = 0, \quad c = p.$$

The recurrence squares the complex conjugate of the current value:

$$z_{n+1} = \overline{z_n}^2 + c.$$

The Tricorn set is therefore

$$T = \{c \in \mathbb{C} : z_0 = 0, z_{n+1} = \overline{z_n}^2 + c, (z_n) \text{ remains bounded}\}.$$

10.2 B: TMDS

The use of dual-wired differential signals allows for the original signal to be split and sent between two wires where one of the signals is inverted.

That is:

$$V_+(t) = V_{cm} + \frac{V_s(t)}{2} + V_n(t)$$

$$V_-(t) = V_{cm} - \frac{V_s(t)}{2} + V_n(t)$$

Where V_{cm} is a common-mode DC voltage, V_s signal voltage and V_n the noise.

A differential amplifier is used to measure the voltage difference between the two pins (as opposed to ground which is what is typically used)

Hence:

$$\begin{aligned} V_{out}(t) &= V_+(t) - V_-(t) \\ \Rightarrow V_{out}(t) &= (V_+(t) = V_{cm} + \frac{V_s(t)}{2} + V_n(t)) - (V_-(t) = V_{cm} - \frac{V_s(t)}{2} + V_n(t)) \\ &\Rightarrow V_{out}(t) = V_s(t) \end{aligned}$$

10.3 C: Perturbation Theory

The Perturbation theory is a method typically used in quantum mechanics to analyse complex quantum signals in terms of their simpler variants. It typically involves solving the exact solution of a simpler related problem, and applying that solution to more complex problems after.

We have applied this in our deep-zoom extension as typically zooming into the Mandelbrot set at a depth factor such as 10^{-15} requires at least 60 bits to represent microscopic decimals. This would require computation too heavy for the DSP slices to handle efficiently.

However by calculating a single high-precision reference orbit, Z_n , on the CPU for the centre of the screen C_{ref} , the FPGA is only required to calculate the microscopic distance, ϵ_n of neighbouring pixels with distance δ from the reference orbit.

This is proved through the following:

Taking the standard Mandelbrot equation, $X_{n+1} = X_n^2 + C$, and substituting $X_n = Z_n + \epsilon_n$ and $C = C_{ref} + \delta$:

$$Z_{n+1} + \epsilon_{n+1} = (Z_n + \epsilon_n)^2 + (C_{ref} + \delta)$$

$$\Rightarrow Z_{n+1} + \epsilon_{n+1} = Z_n^2 + 2Z_n\epsilon_n + \epsilon_n^2 + C_{ref} + \delta$$

Given $Z_{n+1} = Z_n^2 + C_{ref}$:

$$\epsilon_{n+1} = 2Z_n\epsilon_n + \epsilon_n^2 + \delta$$

As this δ value is extremely small, if it is passed to hardware, it will instantly underflow and round to 0, we apply a scaled state variable, $\omega_n = \epsilon_n \times S$ where S is a scaled factor equal to $\approx 2^{51}$.

We then pass the scaled orbit array to hardware and to the following calculation:

$$\omega_{n+1} = 2Z_n\omega_n + (\epsilon_n^2 \times S) + \delta_{scaled}$$

(Note that the squared ϵ term is not passed as a scaled factor as the scaled quadratic term includes an S^2 term which is too large)

10.4 D: Redundant Blocks

In making our final v3 release, we had many blocks that ended up being made redundant due to significant alterations in updates to maximise our throughput.

One of these blocks was the original packer block. We originally used this as our 24-bit RGB to 32-bit output stream for AXI4-Stream interface into VDMA. However, given that we now write straight to DDR through the pixel write engine. The packer basically became a part of the write engine but more optimised for our write engine hence becoming redundant.

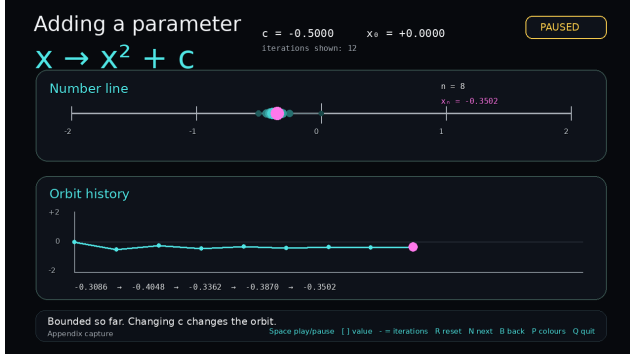
Another block that became redundant was the reorder buffer. This block was used to ensure parallelised computed pixels were ordered in terms of their sequence number before being outputted to the Colour Palette. This was used when we would map pixels through a raster scan. This block became redundant when we had removed it due to its negative effect to our latency mentioned later in the Reorder Buffer Timing Violations.

The final redundant block was our pixel scheduler and its associative blocks. Originally, the pixel scheduler allowed for pixel by pixel scanning to the iteration core block. This was able to compute at a well enough level for standard implementations, but when moving to a more complex design, using M-S algorithms and progressive rendering, this scheduler no longer sufficed and we needed to scan via tiles rather than pixels for efficiency.

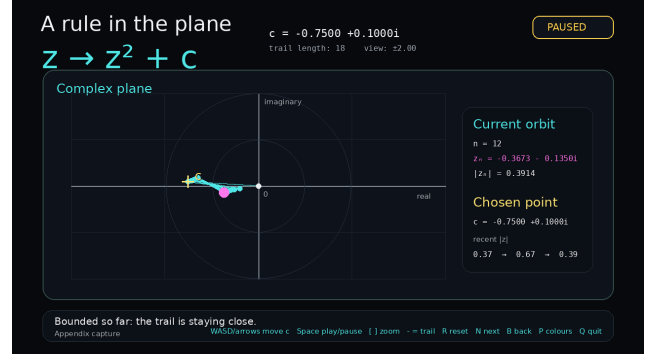
10.5 E: Processing System

10.5.1 Full-PS Scenes

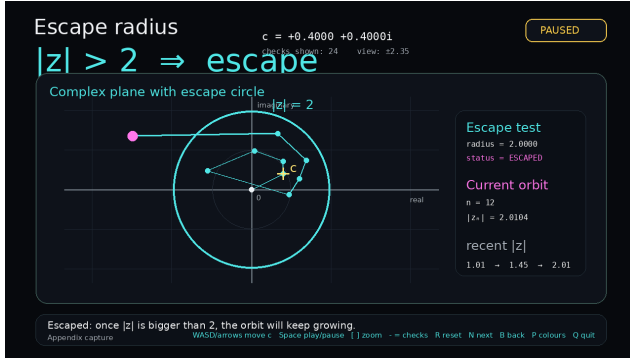
Figure XIX shows the fully processing-system-rendered walkthrough scenes. These scenes were generated entirely on the PS side using Python image composition before the project transitions into PL-backed fractal rendering. The design, however, is not fully finished, and will be streamlined and polished before the final demo.



(a) Scene 1: real-valued recurrence.



(b) Scene 2: complex-plane orbit.



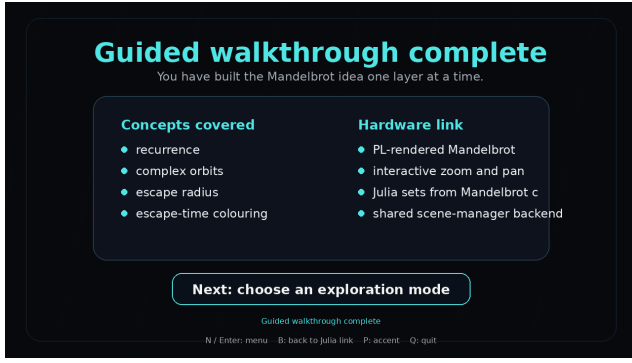
(c) Scene 3: escape-radius test.



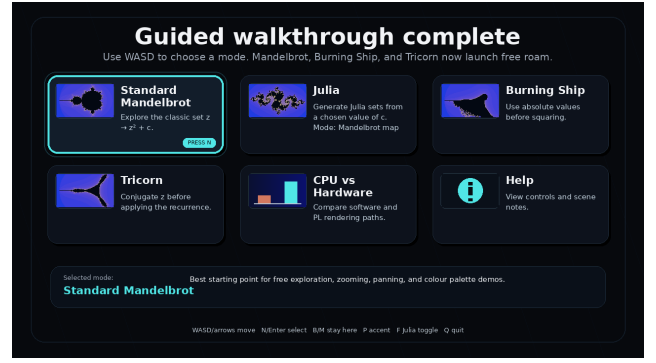
(d) Scene 4: escape-time colouring.

Figure XIX: Fully PS-rendered educational walkthrough scenes.

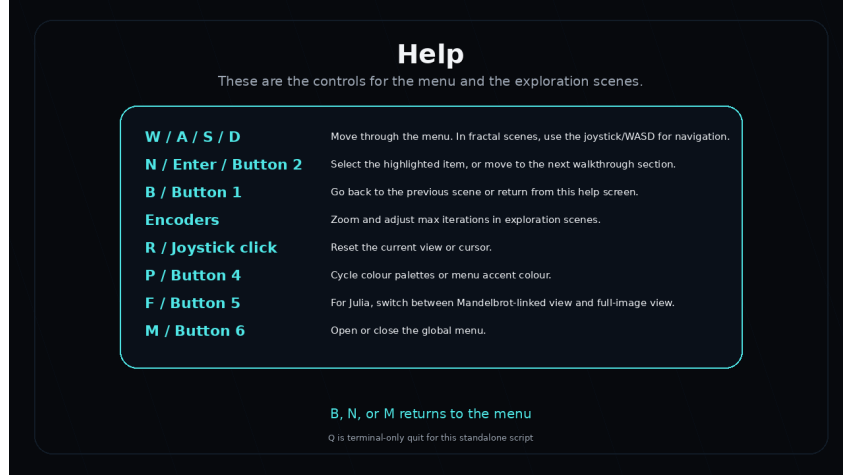
Figure XX shows the PS-rendered application screens used after the guided walkthrough. These screens are also composed on the PS and are used to summarise the tutorial, present the free-roam menu, and display the control help page.



(a) Walkthrough summary screen.



(b) Main free-roam menu.



(c) Controller and keyboard help screen.

Figure XX: Additional PS-rendered application screens.

10.5.2 PL-Backed Scenes

Figure XXI shows the final PL-backed fractal modes available from the FractalScope free-roam menu. In these scenes, the PS handles input, scene state, HUD composition, register programming, framebuffer management, and HDMI commit, while the PL performs the fractal computation and writes the rendered image into DDR.

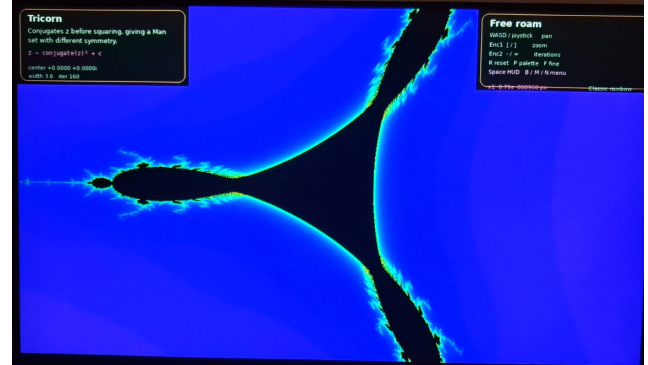
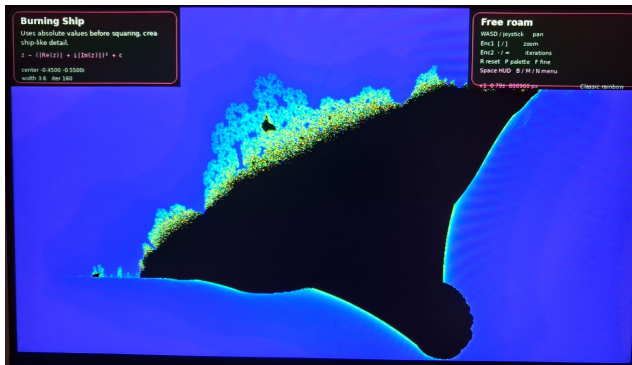
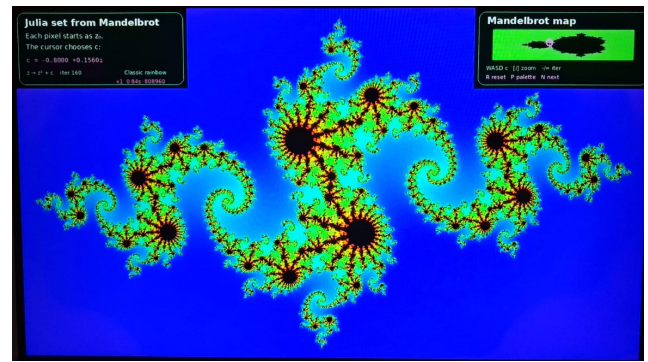
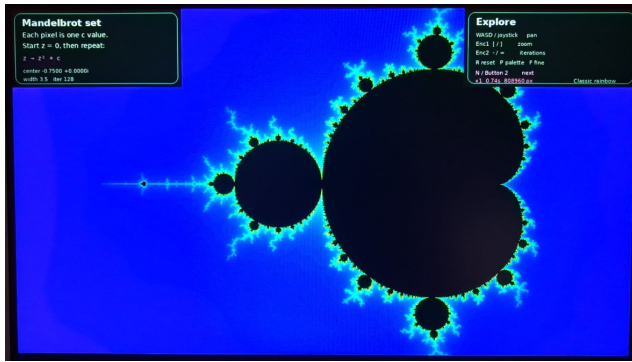


Figure XXI: PL-backed free-roam fractal scenes rendered by the FPGA accelerator.